

Monografia metodologica integrata

Motore genetico strutturale per la pubblicazione forward di zone di ingresso persistenti e condizionate su indici

12 aprile 2026

Indice

1	Premessa editoriale	1
2	Sintesi esecutiva	2
3	Scopo del documento e posizionamento del modello	3
3.1	Problema da risolvere	3
3.2	Perché il modello non è un normale trading system reattivo	3
3.3	Esiti desiderati	3
4	Criterio di integrazione delle fonti e assunzioni recepite	6
4.1	Metodo di consolidamento	6
4.2	Estensioni recepite direttamente	6
4.3	Assunzioni mancanti rese esplicite in questa monografia	7
5	Decisioni normative congelate del baseline integrato	8
5.1	Funzione di questo capitolo	8
5.2	Baseline congelato: tabella delle scelte non negoziabili	8
5.3	Convenzioni operative che il documento assume senza eccezioni	10
5.4	Baseline numerico del layer genetico	10
5.5	Baseline del protocollo di stima	11
5.6	Distinzione tra baseline, catalogo comparativo e ricerca esplorativa	11
6	Hard-lock parametrico, politica univoca di stima e benchmark di robustezza	12
6.1	Obiettivo di hard-lock	12
6.2	Congelamento del criterio di arresto per hypervolume	12
6.3	Congelamento quantitativo di purge ed embargo	13
6.4	Congelamento del bootstrap dipendente	13
6.5	Politica univoca di stima	14
6.6	Finestra ufficiale: expanding default, rolling solo benchmark	14
6.7	cDCC come benchmark large-N e benchmark di coerenza parametrica	14
6.8	Benchmark semiparametrici con shrinkage avanzato: SP-DCC e RSP-DCC	15
6.9	Figure e build documentale congelata	15
7	Formalizzazione probabilistica del dominio	16
7.1	Spazio di probabilità, filtrazione e tempi di evento	16

7.2	Definizione formale del segnale	16
7.3	Zona di ingresso come insieme misurabile prezzo-tempo	17
7.4	Tempi di arresto rilevanti	17
7.5	Evento assorbente di target strutturale	18
8	Contratto del segnale: campi core, invarianti e semantica	19
8.1	Campi core	19
8.2	Invarianti di orientamento	19
8.3	No-refresh come invariabile strutturale	20
8.4	Ragione del core hash	20
9	Semantica degli eventi di mercato	21
9.1	Touch, invalidazione e target con quote conservative	21
9.2	Definizione di raw touch	21
9.3	Gap semantics	21
9.4	Target before entry	22
10	Layer informativo multidimensionale	23
10.1	Tensore di feature	23
10.2	Canali minimi obbligatori	23
10.3	Vettore aleatorio da stimare	23
11	Catalogo dei target strutturali e operatore di generazione dei candidati	25
11.1	Perché il catalogo dei target va formalizzato	25
11.2	Famiglie ammesse del catalogo strutturale	26
11.3	Operatore di scoring strutturale	26
11.4	Regola di selezione del target nel caso di più candidati validi	26
11.5	Coerenza tra target e geometria della zona	27
11.6	Filtri di esclusione del catalogo	27
12	Eteroschedasticità univariata: famiglie, confronto e ruolo nel motore	28
12.1	Scomposizione di base	28
12.2	GARCH(1,1)	28
12.3	EGARCH(1,1)	28
12.4	GJR-GARCH	29
12.5	Realized GARCH	29
12.6	Conclusione metodologica sul layer univariato	29
13	Covarianza condizionale multi-indice	30
13.1	Decomposizione generale	30
13.2	CCC come benchmark	30
13.3	DCC come baseline parsimoniosa	30
13.4	ADCC e correlazioni asimmetriche	30
13.5	BEKK come opzione PSD-robusta	31
13.6	SP-DCC e RSP-DCC come benchmark semiparametrici	31

13.7	Rischio sistemico proiettato	31
13.8	Requisiti di stabilità	31
14	Protocollo di stima, distribuzioni delle innovazioni e politiche di fallback	32
14.1	Mean model e distribuzione delle innovazioni	32
14.2	Policy univoca di stima MLE/QMLE	33
14.3	Protocollo di stima univariato	33
14.4	Diagnostiche minime obbligatorie	33
14.5	Measurement equation per Realized GARCH	34
14.6	Protocollo di stima del layer DCC/ADCC/BEKK	34
14.7	Policy di fallback cross-index	34
14.8	PSD enforcement e regola di rifiuto	35
14.9	Regolarizzazione, shrinkage e robustezza ai collassi di regime	35
15	Generazione della zona, durata e invalidazione	36
15.1	Costruzione target-first	36
15.2	Centro della entry zone	36
15.3	Ampiezza della zona	36
15.4	Durata di validità	37
15.5	Invalidation price	37
15.6	Condizioni minime di emissione	37
16	Survival pre-entry: touch, guardie e competing risks	38
16.1	Perché il touch non è una semplice classificazione binaria	38
16.2	Hazard di raw touch	38
16.3	Guardie pass, soft fail e hard reject	38
16.4	Altri rischi concorrenti pre-entry	39
16.5	Sopravvivenza del segnale pending	39
17	Survival post-entry: target, stop e timeout	40
17.1	Necessità di un secondo blocco	40
17.2	Hazard post-entry	40
17.3	Probabilità end-to-end	40
18	Guardie di regime e persistenza eteroschedastica	41
18.1	Eteroschedasticità come regola di permanenza	41
18.2	Volatility regime break	41
18.3	Guardie soft	41
19	Architettura del motore genetico	42
19.1	Cosa codifica il cromosoma	42
19.2	Inizializzazione della popolazione	42
19.3	Operatori per geni misti	42
19.4	Fitness multi-obiettivo	43
19.5	Vincoli	43

19.6	Regola di dominance vincolata	43
19.7	Criteri di arresto	43
20	Parametrizzazione concreta del motore genetico, domini dei geni e penalizza-	
	zione di complessità	44
20.1	Domini dei geni	44
20.2	Inizializzazione feasible-first	45
20.3	Dominance vincolata e archivio elitista	45
20.4	Penalizzazione di complessità	46
20.5	Criteri di arresto integrati	46
20.6	Selezione finale del bundle frozen	46
21	Macchina a stati runtime	47
21.1	Stati minimi	47
21.2	Precedenza degli eventi	47
21.3	Transizioni principali	48
21.4	Proprietà desiderate della macchina	48
22	Eventi speciali, finestre esecutive e casi limite della macchina a stati	50
22.1	Perché servono casi limite formalizzati	50
22.2	Stato terminale EXECUTION_WINDOW_MISSED	50
22.3	cancel_on_expiry in ORDER_WORKING	50
22.4	Precedenza fill > cancel a timestamp identico	51
22.5	unsupported_event e robustezza del dispatcher	51
22.6	Partial fill come boundary esplicito	51
22.7	Restart e idempotenza degli ordini	52
23	Validazione OOS e controlli anti-overfitting	53
23.1	Split temporali ordinati	53
23.2	Purge ed embargo	53
23.3	Nested walk-forward	53
23.4	Bootstrap dipendente e inferenza robusta	54
23.5	DSR e PBO	54
23.6	White Reality Check e Hansen SPA	54
24	Protocollo di validazione concreto: DSR, PBO, tassonomia dei regimi e gate	
	decisionali	55
24.1	Topologia del nested walk-forward	55
24.2	Purge, embargo e fit dei regimi	55
24.3	PSR e DSR	56
24.4	Numero effettivo di prove	56
24.5	PBO tramite CSCV/CPCV	56
24.6	White Reality Check e Hansen SPA come layer research-only	57
24.7	Gate di fold e decisione globale	57
24.7.1	Gate di fold	57

24.7.2	Decisione globale	57
24.8	Contratto minimo dei report	57
25	Metriche operative e reporting per regime	59
25.1	Metriche lifecycle	59
25.2	Metriche economiche e di rischio	59
25.3	Calibrazione probabilistica	60
25.4	Tassonomia dei regimi	60
25.5	Esperimenti di robustezza	60
26	Reproducibilità, bundle frozen e industrializzazione	61
26.1	Problema generale	61
26.2	Artifact obbligatori	61
26.3	Perché è essenziale	61
26.4	Relazione con la no-refresh doctrine	62
27	Testabilità e tracciabilità	63
27.1	Classi minime di test	63
27.2	Tracciabilità requisito -> test	63
28	Architettura finale del motore come sintesi concettuale	64
29	Conclusione	65
30	Boundary implementativi: da specifica a motore eseguibile	66
30.1	Separazione dei layer	66
30.2	Confine fra research semantics e runtime semantics	66
30.3	Frozen bundle e versionamento	67
30.4	Ledger delle decisioni	67
31	Protocollo operativo dalla ricerca al run controllato	68
31.1	Sequenza corretta	68
31.2	Errori da evitare	68
31.3	Candidate-live gate	68
32	Pseudocodice di alto livello	70
32.1	Training	70
32.2	Emission inference	70
32.3	Runtime	71
33	Scheletro della suite di test	72
33.1	Suite dati e semantica	72
33.2	Suite quantitativa	72
33.3	Suite runtime	72
33.4	Suite validation	72

34 Decisioni che questa monografia congela in modo definitivo	74
A Glossario dei campi e dei simboli	75
A.1 Campi principali del segnale	75
A.2 Stati principali	75
B Catalogo sintetico dei requisiti atomici	77
B.1 Requisiti dati e semantica	77
B.2 Requisiti modello multidimensionale	77
B.3 Requisiti volatilità/covarianza	77
B.4 Requisiti motore genetico	78
B.5 Requisiti persistenza ed esecuzione	78
C Matrice sintetica delle metriche	79
D Contratto minimo di industrializzazione	80
E Appendice dimostrativa hard-locked: lemmi, teoremi, proposizioni e sketch di dimostrazione	81
E.1 Proposizione 1 — Il tempo di raw touch è uno stopping time	81
E.2 Proposizione 2 — Minimo di tempi terminali concorrenti	81
E.3 Proposizione 3 — Non negatività dello stress sistemico proiettato	82
E.4 Proposizione 4 — Monotonia della semilarghezza della zona	82
E.5 Proposizione 5 — Invarianza del core sotto retraining forward-only	82
E.6 Proposizione 6 — Probabilità end-to-end come decomposizione di survival	82
E.7 Proposizione 7 — Well-definedness del DCC baseline	83
E.8 Proposizione 8 — BEKK e positiva definitezza strutturale	83
E.9 Lemma 1 — Compattezza dello spazio dei cromosomi feasible	83
E.10 Teorema 1 — Esistenza del fronte Pareto feasible e di almeno un bundle congelabile	83
E.11 Lemma 2 — Dipendenza piecewise-Lipschitz della geometria della zona	84
E.12 Teorema 2 — Non-degenerazione minima del landscape di fitness	84
E.13 Corollario 1 — Significato operativo del plateau di hypervolume	85
F Piano di test eseguibile completo e criteri di accettazione	86
F.1 Obiettivo del piano di test	86
F.2 Struttura minima della suite	86
F.3 Fixture obbligatorie	87
F.4 Catalogo scenari Given / When / Then	87
F.4.1 Suite dati e semantica	87
F.4.2 Suite quantitativa e layer di rischio	88
F.4.3 Suite runtime e macchina a stati	88
F.4.4 Suite survival e rischi concorrenti	89
F.4.5 Suite validazione e anti-overfitting	89
F.4.6 Suite frozen bundle e riproducibilità	90
F.5 Traduzione minima in pytest	90

F.6	Criteri di accettazione del package	91
G	Contratti API, topologia del package e ordine dei calcoli	92
G.1	Package finale del motore	92
G.2	Oggetti dati principali	93
G.3	Contratti funzione del layer training	94
G.4	Ordine dei calcoli nel training	94
G.5	Contratti funzione del layer di emissione	95
G.6	Contratti funzione del runtime	95
G.7	Contratti funzione del layer validation	95
G.8	Matrice delle dipendenze tra moduli	96
H	Schemi machine-readable, ledger e reporting unificato	97
H.1	Frozen manifest JSON	97
H.2	Schema record del segnale persistito	97
H.3	Schema minimale del ledger runtime	98
H.4	Schema del report di fold	98
H.5	Schema del report globale	98
H.6	Template minimo del report regime-by-regime	99
I	Casi di failure che la specifica esclude esplicitamente	100
J	Manifest del frozen bundle, lessons learned e contratto di industrializzazione forte	101
J.1	Manifest obbligatorio del bundle frozen	101
J.2	Lezione industriale incorporata nella specifica	102
J.3	Hash chain e ledger	102
J.4	Contratto di replay deterministico	102
J.5	Boundary tra package scientifico e motore operativo	102
J.6	Manifest documentale dei diagrammi e della build	103
K	Riferimenti tecnici prioritari	104
K.1	Bibliografia puntuale delle estensioni hard-locked	104
K.2	Famiglie metodologiche coperte dal documento	105
L	Nota d'uso del documento	106

Monografia metodologica integrata

Motore genetico strutturale per la pubblicazione forward di zone di
ingresso persistenti e condizionate su indici

Versione 6.0 — theoretical hard-locked

Natura del documento

Specifica tecnico-metodologica unificata, quantitativa, runtime-ready e orientata alla validazione OOS.

Fonti integrate

PDF base, monografie integrate precedenti, Deep Research Backing Conversation e formalizzazioni SRS, quantitative, runtime, validation, test e packaging rifuse in un unico corpo unico senza dipendenze da documenti esterni.

Uso previsto

Baseline ufficiale per ricerca, implementazione, audit, validazione, industrializzazione e deployment controllato.

Principio architetturale centrale

Il live può veto-are l'esecuzione al touch, ma non può riscrivere il core del segnale.

12 aprile 2026

Documento sostitutivo creato come nuovo artefatto unitario, con hard-lock parametrico, appendice dimostrativa e build documentale congelata.

Premessa editoriale

Questo documento sostituisce la precedente sintesi breve e riporta il modello in una forma coerente con la difficoltà reale del problema. Il punto di partenza non è una strategia discrezionale né un classificatore direzionale tradizionale: l'oggetto centrale è un **contratto quantitativo persistente** che pubblica ex ante una zona di ingresso futura, una zona target e una finestra di validità definita simultaneamente da prezzo, tempo e regime di rischio.

La monografia integra in un unico corpo logico quattro famiglie di contenuti che nei sorgenti comparivano in modo separato o conversazionale:

1. la SRS normativa del segnale persistente, con campi, invarianti, stati e tracciabilità;
2. la formalizzazione quantitativa del layer prezzo-volatilità-covarianza;
3. la decomposizione del problema decisionale in survival pre-entry e post-entry;
4. il protocollo di validazione anti-overfitting necessario perché un motore genetico non degeneri in un cercatore del miglior backtest casuale.

La tesi architetture di fondo è semplice ma radicale: **il live può veto-are l'esecuzione al touch, ma non può riscrivere il core del segnale**. Da questa assunzione discendono il vincolo di no-refresh, l'idea di core hash, la necessità di una macchina a stati deterministica e la separazione netta fra pubblicazione del contratto e decisione esecutiva.

Sintesi esecutiva

Il modello viene formalizzato come un motore genetico strutturale che opera su indici e contesti multi-indice. La sua funzione non è decidere “entra ora”, bensì scegliere, tra molte architetture candidate, quelle che sanno pubblicare in anticipo zone future persistenti e condizionate con probabilità sufficiente di diventare eseguibili e di raggiungere un target strutturale.

La pubblicazione è sottoposta a cinque vincoli non negoziabili:

- **validità prezzo-tempo:** la zona esiste solo entro un intervallo di prezzo e una finestra temporale deterministica;
- **no-refresh del core:** entry zone, target zone, invalidazione hard e finestra di validità sono immutabili dopo l’emissione;
- **decisione solo al touch reale:** non esiste esecuzione anticipata, predittiva o inferita da gap senza path osservato;
- **eteroschedasticità esplicita:** la varianza condizionale non è decorativa, ma entra nella generazione della zona, nella sua ampiezza, nella durata e nella sua persistenza;
- **covarianza condizionale cross-index:** il rischio non è solo locale; il sistema considera dipendenze dinamiche fra indici e può invalidare un segnale quando il regime di correlazione/covarianza si rompe.

Questa struttura implica che il motore genetico non debba ottimizzare direttamente coefficienti econometrici istantanei, ma **architetture:** subset di feature, classi modello, quantili, soglie di fattibilità, guardie runtime, politiche di conflitto e criteri di validazione. La fitness, per essere epistemicamente difendibile, deve essere multi-obiettivo e valutata solo fuori campione con protocolli time-ordered, purge/embargo, bootstrap dipendente e controlli di selection bias come DSR e PBO.

Scopo del documento e posizionamento del modello

3.1 Problema da risolvere

Il problema affrontato dal motore può essere espresso nel modo seguente: dato un insieme di indici, un'informazione osservata filtrata nel tempo e un catalogo di target strutturali candidati, il sistema deve decidere se abbia senso pubblicare un segnale persistente tale che:

- la zona di ingresso sia futura e non banale;
- il prezzo possa plausibilmente raggiungerla entro un orizzonte finito;
- l'eventuale esecuzione, qualora il prezzo la tocchi, rimanga coerente con il regime di rischio;
- esista una probabilità end-to-end sufficientemente alta di raggiungere il target strutturale prima dei terminal event concorrenti.

Il problema quindi non è classificare il segno del prossimo rendimento. È un problema di **pubblicazione di contratti runtime condizionati**, dove la semantica dell'oggetto emesso deve restare stabile anche dopo il retraining del modello.

3.2 Perché il modello non è un normale trading system reattivo

Un trading system reattivo tradizionale ragiona tipicamente in termini di entry immediata, eventuale trailing, refresh continuo di livelli e modifiche tattiche live. Qui l'impostazione è opposta. La sequenza corretta è:

pubblica -> congela -> monitora -> esegui oppure scadi/invalida

Questa sequenza impedisce che il motore migliori artificialmente la propria performance spostando le condizioni operative in inseguimento del mercato. Dal punto di vista metodologico, il vincolo di no-refresh è un presidio anti-overfitting tanto importante quanto i controlli statistici OOS.

3.3 Esiti desiderati

L'architettura cerca quattro esiti simultanei:

1. pubblicare segnali rari ma strutturalmente coerenti;
2. massimizzare la probabilità di touch eseguibile, non solo di touch grezzo;
3. massimizzare la probabilità di target dato fill, non solo dato emissione;
4. mantenere stabilità cross-regime e cross-index senza dipendere da ricalibrazioni opportunistiche.

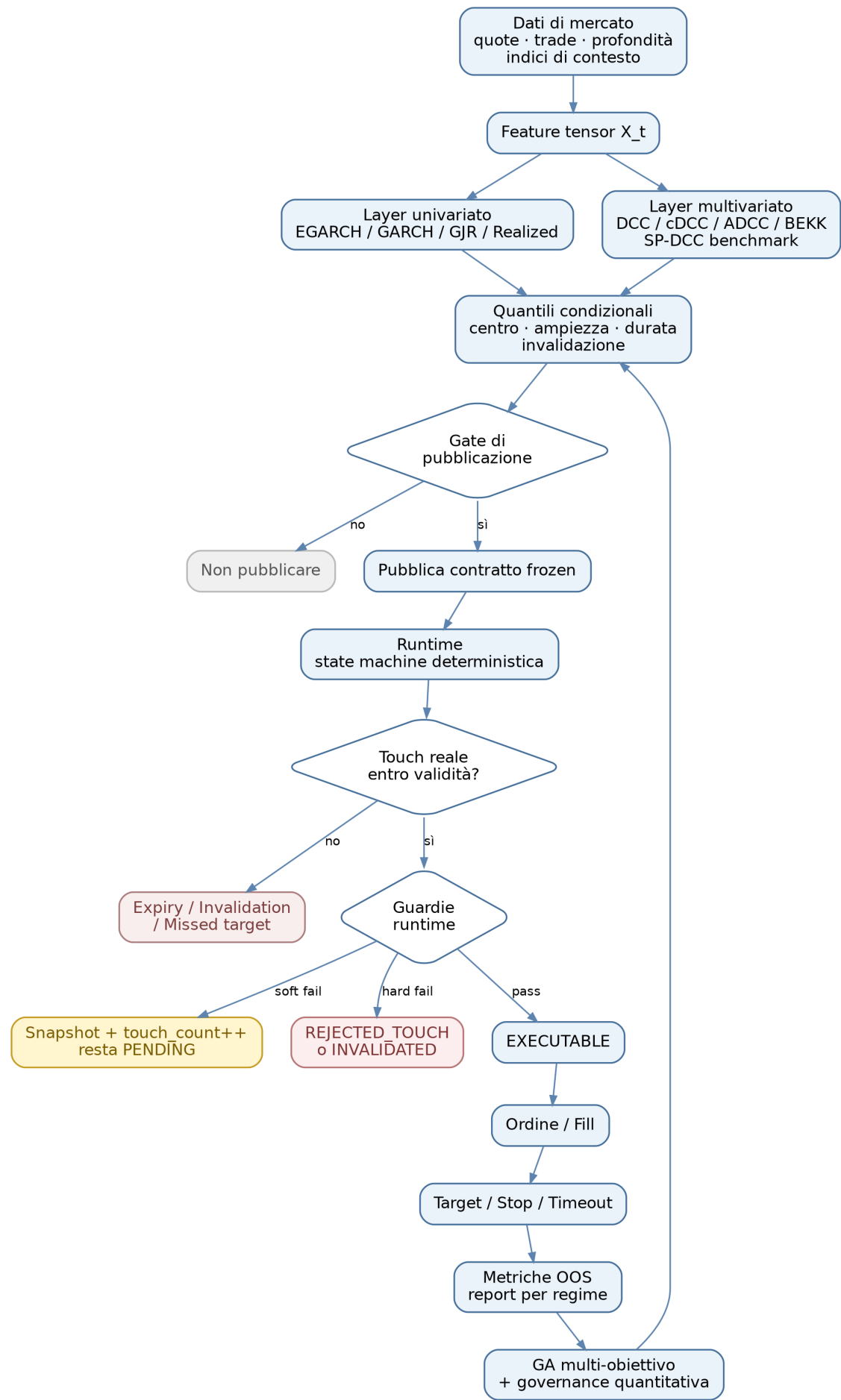


Figura 3.1: Figura 1 — Architettura end-to-end del motore genetico strutturale. La fonte canonica del diagramma è conservata nel bundle in sintassi Mermaid; questa build integra l’asset statico congelato equivalente.

Criterio di integrazione delle fonti e assunzioni recepite

4.1 Metodo di consolidamento

Le fonti disponibili presentavano due forme diverse: una forma SRS più normativa e una forma di deep research più argomentativa, comparativa e progettuale. Il consolidamento effettuato qui segue tre regole:

- ciò che era requisito atomico o semantica vincolante viene mantenuto come norma;
- ciò che era estensione metodologica ben motivata viene promosso a parte integrante del baseline;
- ciò che era conversazionale, ridondante o dipendente dal contesto della chat viene eliminato oppure riformulato come assunzione esplicita.

4.2 Estensioni recepite direttamente

Sono state recepite come parte nativa del modello, e non come note opzionali, le seguenti estensioni:

- interpretazione del segnale come contratto runtime auditabile;
- lettura del motore genetico come selettore di architetture, non di coefficienti estemporanei;
- uso di EGARCH come baseline preferibile rispetto a GARCH semplice in presenza di asimmetrie;
- uso di DCC come baseline parsimoniosa per la correlazione dinamica e considerazione di ADCC/BEKK come famiglie di confronto;
- modellazione del touch come survival problem con competing risks;
- distinzione fra raw touch, touch eseguibile, touch con soft fail e hard reject;
- uso di nested walk-forward, purge/embargo, stationary bootstrap e filtri DSR/PBO per il controllo anti-overfitting;
- introduzione della nozione di bundle frozen come requisito di industrializzazione e riproducibilità.

4.3 Assunzioni mancanti rese esplicite in questa monografia

Poiché i sorgenti non normavano tutti gli aspetti implementativi, questa monografia fissa in più:

- criteri di inizializzazione della popolazione GA;
- famiglia consigliata di operatori per geni misti continui/discreti;
- criteri di arresto evolutivo;
- distinzione tra layer di ricerca e layer di deployment;
- requisito di serializzazione completa degli artifact necessari a ricostruire l'inferenza congelata.

Decisioni normative congelate del baseline integrato

5.1 Funzione di questo capitolo

Le fonti originarie convergevano su una stessa architettura, ma lasciavano ancora alcuni gradi di libertà implementativi che, se mantenuti aperti, avrebbero prodotto una nuova forma di ambiguità: una monografia elegante ma non davvero chiusa. Questo capitolo svolge quindi una funzione deliberatamente normativa. Non introduce un “futuro lavoro”; congela il **baseline operativo** che il resto del documento assume come riferimento, lasciando le famiglie alternative solo come benchmark o classi comparative.

La regola di lettura è la seguente: quando in capitoli successivi compaiono famiglie alternative, esse vanno intese come opzioni del *catalogo di confronto* del motore genetico; quando qui viene definita una scelta di baseline, quella scelta vale come default normativo della specifica integrata.

5.2 Baseline congelato: tabella delle scelte non negoziabili

Blocco	Decisione congelata	Razionale
Clock e granularità	Decisione evento-driven su quote/trade con timestamp UTC ad alta precisione	evita inferenze latenti su gap e allinea ricerca e runtime
Mean model	baseline ARX parsimoniosa a bassa dimensione; fallback zero-mean se i regressori non aggiungono potere esplicativo stabile	separa la previsione del livello medio dalla dinamica della varianza senza gonfiare il modello

Blocco	Decisione congelata	Razionale
Distribuzione innovazioni	Student-t standardizzata come default; skew-t ammessa solo se produce miglioramento robusto e stabile	code pesanti e asimmetrie incidono su CVaR, hazard e calibrazione
Volatilità univariata	EGARCH(1,1) come baseline; GARCH(1,1) come benchmark parsimonioso; GJR e Realized GARCH come famiglie comparative	il problema richiede asimmetrie di volatilità e persistenza non lineare
Covarianza cross-index	DCC(1,1) come baseline; CCC come fallback semplice; ADCC come estensione di stress; BEKK come audit/benchmark PSD-robusto	mantiene parsimonia con più indici e consente rischio sistemico dinamico
Generatore di target	catalogo finito di target strutturali derivato da operatori strutturali espliciti su più orizzonti	evita target “inventati” ex post o dipendenti da trailing live
Geometria della zona	centro da quantile condizionale, ampiezza monotona in volatili- tà/covarianza/spread/illiquidità, durata da quantile di tempo-al-touch	impone eteroschedasticità e validità prezzo-tempo nel contratto
Survival pre-entry	discrete-time hazard complementary log-log con rischi concorrenti	rispetta la natura grouped/event-driven del problema
Survival post-entry	secondo blocco hazard su target/stop/timeout dopo fill	separa correttamente pre-esecuzione e post-esecuzione
Ottimizzazione evolutiva	NSGA-II con dominance vincolata e archivio elitista	preserva fattibilità e non collassa tutto in un solo score
Anti-overfitting	nested walk-forward, purge/embargo, DSR, PBO-CSCV/CPCV, bootstrap dipendente	controlla leakage, multiple proving e selection bias
Runtime	contratto congelato, no-refresh, state machine deterministica, un solo ordine per segnale	impedisce adattamento opportunistico live

Blocco	Decisione congelata	Razionale
Deployability	frozen bundle completo e ricostruibile obbligatorio	senza bundle frozen non esiste OOS forte né audit serio

5.3 Convenzioni operative che il documento assume senza eccezioni

Il baseline integrato assume che:

1. tutti i segnali siano generati e valutati su una semantica causale rigorosa, cioè senza alcun uso di informazione non disponibile al tempo di emissione;
2. il medesimo motore di replay della macchina a stati sia usato sia in validazione sia in runtime;
3. nessun miglioramento di performance ottenuto spostando zone, target o invalidazioni dopo l'emissione sia considerato metodologicamente ammissibile;
4. le famiglie alternative siano usate solo come sorgente di competizione architetturale o benchmark, non come grimaldello per riscrivere la semantica di base del sistema.

5.4 Baseline numerico del layer genetico

Per chiudere anche i dettagli più “ingegneristici”, il baseline integrato adotta i seguenti valori di default, da intendersi come impostazione di riferimento replicabile e non come verità universale del problema:

- popolazione iniziale: 128 cromosomi;
- archivio elitista: 64 soluzioni non dominate;
- tournament size: 2;
- crossover discreto: uniforme a blocchi con probabilità 0.9;
- crossover continuo: simulated binary crossover con $\eta_c = 15$;
- mutazione continua: polynomial mutation con $\eta_m = 20$ e tasso per-gene inversamente proporzionale al numero di geni continui;
- mutazione categorica: tasso 0.05 sui geni di scelta famiglia modello;
- limite generazioni: $G_{max} = 150$;
- criterio hypervolume: obiettivi normalizzati su $[0, 1]^m$, reference point $r = (1.05, \dots, 1.05)$, $\epsilon_{HV} = 10^{-4}$, $G_{stall} = 15$;
- early stop addizionale: peggioramento del DSR mediano di outer-valid per 5 generazioni consecutive con hypervolume inner ancora crescente;
- stop per saturazione infeasible: oltre l'80% della popolazione infeasible per 10 generazioni consecutive;
- complessità massima: vincolo su numero di feature, numero di guardie e presenza simultanea di famiglie altamente parametriche.

Questi numeri non eliminano il ruolo del motore genetico; eliminano piuttosto l'arbitrarietà di implementazione. Un altro baseline numerico può essere studiato, ma deve essere trattato come

variante esplicita e non come comportamento implicito del motore.

5.5 Baseline del protocollo di stima

Il documento congela anche il protocollo di stima minimo:

- stima dei modelli univariati su **expanding window** come default ufficiale, con warm-up minimo $T_{warm} = 750$ osservazioni;
- rolling window ammessa solo come benchmark dichiarato con ampiezza congelata $T_{roll} = 1500$;
- stima dei modelli univariati con policy `T_MLE_FIRST` $\rightarrow$ `QMLE_FALLBACK` e ragione del fallback persistita nel bundle frozen;
- standardizzazione dei residui prima di ogni stima multivariata della correlazione/covarianza;
- stima della dinamica DCC/ADCC/cDCC in due stadi con QMLE su residui standardizzati e matrice di lungo periodo shrinkata quando necessario;
- stima di BEKK come benchmark di audit con QMLE, mai come fallback silenzioso;
- stima dei modelli di quantile e dei modelli di hazard solo su dati causalmente disponibili nella finestra di training del fold;
- nessun cutpoint di regime, soglia di reporting o regola di gating può essere fit sul test del fold;
- ogni bundle di inferenza deve contenere abbastanza artefatti da ricostruire esattamente le trasformazioni che hanno prodotto la zona.

5.6 Distinzione tra baseline, catalogo comparativo e ricerca esplorativa

Per evitare ogni ambiguità, il documento distingue tre livelli:

- **baseline normativo**: ciò che viene congelato qui e che definisce la forma ufficiale del modello;
- **catalogo comparativo**: famiglie che il GA può selezionare o usare come benchmark entro vincoli definiti;
- **ricerca esplorativa**: estensioni che possono essere provate in laboratorio, ma che non fanno parte della specifica ufficiale finché non vengono promosse a baseline.

Questa tripartizione impedisce il problema tipico dei progetti quantitativi complessi: avere un documento “ufficiale” che descrive una cosa mentre il codice, i notebook e l’uso reale ne descrivono un’altra.

Hard-lock parametrico, politica univoca di stima e benchmark di robustezza

6.1 Obiettivo di hard-lock

Il documento è già chiuso su molti aspetti che in progetti quantitativi analoghi restano impliciti: contratto del segnale, no-refresh, equivalenza tra research semantics e runtime semantics, requirement di bundle frozen, selezione multi-obiettivo e validazione OOS. Restavano però alcune libertà implementative che, se lasciate aperte, avrebbero continuato a produrre varianti semanticamente compatibili ma numericamente non equivalenti. Questo capitolo chiude quelle aperture e definisce la **versione theoretical hard-locked** del modello.

La regola è semplice: da qui in avanti ogni componente che influisce su risultati, confronto tra candidati o riproducibilità del bundle deve avere una parametrizzazione congelata. Le famiglie alternative rimangono nel catalogo comparativo del motore genetico, ma il documento non consente più che soglie, finestre, policy di stima o parametri del protocollo OOS vengano lasciati al gusto dell'implementatore.

6.2 Congelamento del criterio di arresto per hypervolume

Sia $\mathcal{A}_g$ l'archivio feasible non dominato alla generazione g e sia $HV_g = HV(\mathcal{A}_g; r)$ l'hypervolume calcolato sul vettore degli obiettivi **normalizzati** nel cubo $[0, 1]^m$. Il riferimento congelato è:

$$r = (1.05, \dots, 1.05) \in \mathbb{R}^m.$$

Il criterio di plateau non è più lasciato implicito. La build hard-locked fissa:

$$\epsilon_{HV} = 10^{-4}, \quad G_{stall} = 15.$$

Si dichiara plateau dell'hypervolume se e solo se:

$$\max_{j=g-G_{stall}+1, \dots, g} |HV_j - HV_{j-1}| \leq \epsilon_{HV}.$$

Il criterio di arresto evolutivo assume quindi forma deterministica:

1. limite massimo di generazioni $G_{max} = 150$;
2. stop per plateau se la condizione precedente vale per $G_{stall} = 15$ generazioni consecutive;
3. early stop addizionale se il DSR mediano di outer-valid peggiora per 5 generazioni consecutive mentre l'hypervolume inner continua a migliorare;
4. arresto forzato se oltre l'80% della popolazione risulta infeasible per 10 generazioni consecutive.

6.3 Congelamento quantitativo di purge ed embargo

Sia H_{label}^{max} l'orizzonte massimo, espresso nell'unità temporale canonica del fold (barre o bucket evento-driven), con cui qualunque label, target, timeout, guardia o feature può ancora influenzare un esito del segnale. Il documento hard-locked congela:

$$L_{purge} = H_{label}^{max}$$

e

$$L_{emb} = \max \left(\lceil 0.01 N_{fold} \rceil, \lceil 0.25 H_{label}^{max} \rceil \right),$$

dove N_{fold} è la numerosità del fold nella stessa unità temporale canonica. In termini operativi:

- il purge elimina dal training tutti i punti che possono contaminare causalmente il blocco di valid/test;
- l'embargo protegge i bordi temporali anche quando l'overlap non è completamente catturato dal purge puro;
- nessun esperimento presentato come baseline può usare purge o embargo disattivati.

6.4 Congelamento del bootstrap dipendente

Per l'inferenza robusta e per la costruzione di intervalli su metriche serialmente dipendenti, il baseline fissa lo stationary bootstrap con:

$$\bar{\ell} = \max \left(10, \lceil 1.5 H_{label}^{max} \rceil \right), \quad p_{SB} = 1/\bar{\ell}, \quad B = 2000.$$

Qui $\bar{\ell}$ è la lunghezza media dei blocchi geometrici e B è il numero di replicazioni bootstrap. La build hard-locked usa quindi:

- stationary bootstrap come default per metriche dipendenti;
- HAC/Newey–West come supporto per confronti lineari semplici;

- intervalli al 95% come baseline di reportistica, con eventuali correzioni BCa classificate come **research-only**.

6.5 Politica univoca di stima

Il documento congela la seguente politica unica:

$$\text{POLICY-EST-001} = \text{T_MLE_FIRST} \rightarrow \text{QMLE_FALLBACK}.$$

La regola si applica in modo deterministico:

1. i modelli univariati della volatilità vengono stimati con **MLE Student-t standardizzata** come prima scelta;
2. il fallback a **Gaussian QMLE** è ammesso solo se si verifica almeno una delle condizioni seguenti:
 - mancata convergenza dell'ottimizzatore entro 250 iterazioni;
 - Hessiana non invertibile o non positiva definita a convergenza;
 - gradi di libertà stimati $\nu \leq 2.1$;
 - violazione dei vincoli strutturali della famiglia o diagnostiche minime non soddisfatte dopo un solo riavvio deterministico dell'ottimizzazione;
3. i modelli DCC, ADCC e cDCC sono stimati in due stadi con **QMLE su residui standardizzati**;
4. BEKK viene mantenuto come famiglia di audit/benchmark ed è stimato in QMLE;
5. il bundle frozen deve persistere esplicitamente sia la policy scelta sia la ragione del fallback.

In assenza di una ragione di fallback persistita, il run viene trattato come non ricostruibile in senso forte.

6.6 Finestra ufficiale: expanding default, rolling solo benchmark

Il documento ammetteva sia rolling sia expanding window. La versione hard-locked congela:

- **expanding window** come default ufficiale del baseline;
- warm-up minimo $T_{warm} = 750$ osservazioni canoniche prima del primo fold valutabile;
- **rolling window** ammessa solo come benchmark esplicito con ampiezza congelata $T_{roll} = 1500$.

Questa scelta riduce il numero di gradi di libertà nella stima e rende comparabili i report di fold. Ogni tabella o report che non usi expanding baseline deve dichiararlo esplicitamente come variante comparativa.

6.7 cDCC come benchmark large-N e benchmark di coerenza parametrica

Per sistemi con numero di indici elevato o quando l'interpretazione dei parametri del DCC classico è metodologicamente delicata, il documento hard-locked impone di includere **cDCC** come

benchmark di robustezza. Il cDCC non sostituisce il baseline DCC, ma viene usato come controllo quando:

- $N \geq 8$ indici nel contesto cross-index;
- la stabilità della matrice R_t è critica per la permanenza del segnale;
- il DCC classico mostra forte sensibilità alla scelta del campione o ai residui standardizzati.

La build non congela cDCC come default, ma congela l'obbligo di considerarlo come benchmark quando il problema entra in regime **large-N** o di elevata fragilità parametrica.

6.8 Benchmark semiparametrici con shrinkage avanzato: SP-DCC e RSP-DCC

Quando il numero di indici cresce ulteriormente o quando la struttura della covarianza condizionale diventa fragile per collinearità, il documento integra anche benchmark **semiparametrici** e **regularized semiparametrici** del tipo SP-DCC / RSP-DCC. Questi benchmark non sostituiscono la baseline DCC/ADCC, ma servono a verificare se:

- una stima con shrinkage più aggressivo migliora la stabilità degli autovalori di H_t ;
- le decisioni di publish gate o di invalidazione di regime sono troppo sensibili alla sola parametrizzazione DCC standard;
- il vantaggio apparente del modello parametrico dipende da una fragilità numerica e non da vero contenuto informativo.

6.9 Figure e build documentale congelata

I diagrammi logici del motore vengono mantenuti anche in forma **Mermaid canonica** nel bundle documentale. La build distribuita integra nel PDF le **figure statiche equivalenti** già congelate, in modo da evitare differenze build-to-build dovute al renderer grafico.

La versione di renderer target congelata per eventuali rebuild documentali native è:

- mermaid@11.14.0
- theme: **neutral**
- orientamento logico: top-down
- sorgente canonica dei diagrammi: **.mmd** nel bundle
- asset di build integrato nel PDF: **.png** statico congelato

Questa scelta non introduce dipendenze esterne nel documento distribuito, ma rende verificabile e ricostruibile anche la parte grafica della specifica.

Formalizzazione probabilistica del dominio

7.1 Spazio di probabilità, filtrazione e tempi di evento

Sia $(\Omega, \mathcal{F}, \mathbb{P})$ uno spazio di probabilità completo. Sia $(\mathcal{F}_t)_{t \geq 0}$ una filtrazione destra continua che rappresenta l'informazione disponibile fino al tempo t .

Le osservazioni di mercato non sono ridotte a barre regolari. In generale, si ammette una sequenza di tempi di evento $(T_n)_{n \geq 1}$ dove ogni T_n corrisponde a una quote update, a un trade, a un tick ricostruito o a un evento di clock di sistema. L'uso di tempi di evento è cruciale perché il concetto di touch, invalidazione e target non può essere affidato a semantiche bar-based quando si vogliono evitare inferenze latenti sui gap.

Per ogni indice i si osservano processi di prezzo, bid, ask, mid e volumi di profondità. Si osserva inoltre un vettore multi-indice dei rendimenti:

$$r_t = (r_{1,t}, \dots, r_{N,t})^\top.$$

7.2 Definizione formale del segnale

Un segnale persistente emesso al tempo t_0 è una variabile aleatoria S_{t_0} misurabile rispetto a $\mathcal{F}_{t_0}$ definita come tupla:

$$S_{t_0} = (\text{id}, i, s, Z_e, Z_g, I, [t_{\min}, t_{\max}], \Gamma, H_{t_0}, X_{t_0}, \chi, h_c)$$

dove:

- i è l'indice bersaglio;
- $s \in \{+1, -1\}$ è il lato operativo;
- Z_e è la zona di ingresso;
- Z_g è la zona target;
- I è la barriera hard di invalidazione pre-entry;

- $[t_{\min}, t_{\max})$ è la finestra temporale di validità;
- Γ rappresenta la configurazione di runtime guards;
- H_{t_0} è lo snapshot della covarianza condizionale;
- X_{t_0} è il tensore di feature all'emissione;
- χ è il cromosoma/architettura che ha generato il segnale;
- h_c è il core hash che congela i campi immutabili.

La definizione come tupla è importante: il segnale non è una probabilità né una label, ma un oggetto con semantica interna e vita runtime.

7.3 Zona di ingresso come insieme misurabile prezzo-tempo

Sia P_t^{touch} il processo di prezzo utilizzato per valutare il touch, dipendente dal lato e dalla sorgente configurata (best ask, best bid, mid, last). La zona di ingresso viene rappresentata come insieme misurabile:

$$\mathcal{Z}_e = \{(p, t) : p \in [e_L, e_H], t \in [t_{\min}, t_{\max})\}.$$

Analogamente, la zona target è:

$$\mathcal{Z}_g = \{(p, t) : p \in [g_L, g_H], t \in [t_{\min}, +\infty)\},$$

fermo restando che il target ha senso solo dopo fill oppure, sul piano analitico pre-entry, come evento concorrente “target reached before entry”.

7.4 Tempi di arresto rilevanti

Si definiscono tempi di arresto rispetto alla filtrazione evento-driven:

- τ_{touch} : primo tempo in cui il prezzo di touch entra in $[e_L, e_H]$ mentre il segnale è in stato pending e il tempo è valido;
- τ_{inv} : primo tempo di invalidazione hard di prezzo o di regime;
- τ_{exp} : tempo di scadenza;
- τ_{miss} : primo tempo in cui il target viene raggiunto prima dell'entry zone;
- τ_{exe} : tempo di eseguibilità effettiva, cioè tempo di raw touch con tutte le guardie soddisfatte;
- τ_{fill} : tempo di fill dell'ordine;
- τ_{tar} : tempo di raggiungimento del target dopo fill.

Questa notazione chiarisce che il problema è naturalmente un problema di stopping times concorrenti.

7.5 Evento assorbente di target strutturale

Il target strutturale non è un semplice prezzo desiderato; è un evento assorbente definito da una regione di completamento. In forma minimale:

$$A_g = \{\omega \in \Omega : \exists t \geq \tau_{fill} \text{ tale che } P_t^{target}(\omega) \in [g_L, g_H]\}.$$

L'assorbimento è concettuale: una volta realizzato l'evento di target, la traiettoria del segnale in relazione allo scopo operativo si considera conclusa. Questa formalizzazione è importante per evitare ambiguità con trailing o target ridefiniti live.

Contratto del segnale: campi core, invarianti e semantica

8.1 Campi core

I campi core rappresentano il contenuto immutabile del contratto operativo. Essi includono almeno:

- identificatore univoco del segnale;
- codice canonico dell'indice;
- lato LONG/SHORT;
- istante di emissione;
- valid from e valid until;
- estremi inferiori e superiori della entry zone;
- estremi inferiori e superiori della target zone;
- invalidation price hard;
- sorgente di prezzo per il touch;
- granularità evento ammessa;
- volatilità condizionale all'emissione;
- snapshot della covarianza condizionale;
- hash delle feature e core hash.

La ragione per cui σ_{issued} e H_{issued} vanno trattati come parte integrante del contratto è che la permanenza del segnale dipende dalla compatibilità del regime corrente con il regime emittente.

8.2 Invarianti di orientamento

Per LONG deve valere:

$$I < e_L \leq e_H < g_L \leq g_H.$$

Per SHORT deve valere:

$$g_L \leq g_H < e_L \leq e_H < I.$$

Questi vincoli impediscono segnali semanticamente contraddittori. Non basta che le quantità siano numericamente ordinate; devono essere coerenti con il lato.

8.3 No-refresh come invariabile strutturale

Il no-refresh non è una policy estetica, ma un invariante:

- se il segnale è in stato pending, executable o order working, nessun campo core può essere modificato;
- il retraining del modello può influenzare solo segnali futuri;
- il runtime può aggiornare soltanto campi di audit e stato, mai la geometria del contratto.

In simboli, se $Core(S)$ denota l'insieme dei campi core del segnale S , allora per ogni tempo $t \geq t_0$ finché il segnale è attivo:

$$Core(S_t) = Core(S_{t_0}).$$

8.4 Ragione del core hash

Il core hash serve a tre funzioni:

1. auditabilità ex post;
2. rilevazione di corruzioni o mutazioni illegittime;
3. ripristino dopo restart senza duplicare semantica.

Il core hash è l'impronta del contratto, non un semplice checksum tecnico.

Semantica degli eventi di mercato

9.1 Touch, invalidazione e target con quote conservative

Per LONG, salvo override persistito:

- touch valutato su best ask;
- invalidazione pre-entry su best bid;
- target su best bid.

Per SHORT, simmetricamente:

- touch valutato su best bid;
- invalidazione pre-entry su best ask;
- target su best ask.

Questa scelta non è arbitraria: è una semantica conservativa che incorpora il lato della microstruttura e impedisce di contare come eseguibile un livello toccato solo da un riferimento favorevole ma non negoziabile.

9.2 Definizione di raw touch

Il raw touch al tempo τ si ha se e solo se:

$$P_{\tau}^{touch} \in [e_L, e_H] \quad \wedge \quad \tau \in [t_{\min}, t_{\max}) \quad \wedge \quad state_{\tau^-} = PENDING.$$

Questo evento non implica ancora eseguibilità. Il raw touch è solo il punto di ingresso nel layer delle guardie.

9.3 Gap semantics

In assenza di ricostruzione intrabar o di path osservato, il sistema non deve inferire un touch latente attraverso un gap. Questa regola è metodologicamente decisiva. Un modello che si autorizzi a inferire path non osservati altera sia la validazione sia la semantica causale dell'oggetto.

9.4 Target before entry

Se il target viene raggiunto prima del raw touch, il segnale entra in stato terminale di missed target. La ragione è causale: il successo non è “prezzo ha visitato una zona compatibile con il target”, ma “prezzo ha prima reso eseguibile il contratto e poi ha completato il percorso verso il target”.

Layer informativo multidimensionale

10.1 Tensore di feature

Per ogni indice bersaglio i e istante di emissione t , si costruisce un tensore:

$$X_t^{(i)} \in \mathbb{R}^{N_I \times N_H \times N_F}$$

dove N_I è il numero di indici nel contesto cross-index, N_H il numero di orizzonti temporali e N_F il numero di feature per coppia indice-orizzonte.

10.2 Canali minimi obbligatori

Il tensore deve includere almeno:

- rendimenti e rendimenti standardizzati;
- realized volatility;
- vol-of-vol;
- spread e proxy di profondità/liquidità;
- misure di stress cross-index;
- correlazioni e spread relativi tra indici;
- descrittori strutturali del target;
- stato di regime;
- sessione, contesto temporale e shock recenti.

Una parte del valore del motore deriva proprio dal fatto che la feature space non è centrata solo sul mercato bersaglio ma sul contesto nel quale quel mercato si muove.

10.3 Vettore aleatorio da stimare

Il vettore minimo da modellare congiuntamente è:

$$Y_t = (\Delta P_e, \Delta P_g, \tau_e, \tau_g, MAE, MFE).$$

Interpretazione:

- ΔP_e : dislocamento atteso fino alla zona di ingresso;
- ΔP_g : dislocamento atteso fino al target;
- τ_e : tempo al touch;
- τ_g : tempo al target;
- MAE : excursus avverso massimo;
- MFE : excursus favorevole massimo.

L'idea chiave è che il modello non produce una singola probabilità isolata. Produce invece un profilo multidimensionale di distanza, tempo e rischio.

Catalogo dei target strutturali e operatore di generazione dei candidati

11.1 Perché il catalogo dei target va formalizzato

Nei documenti sorgente era forte la nozione di **target strutturale**, ma restava solo implicita la procedura con cui i target candidati vengono generati. Questo è un punto metodologicamente cruciale: se la generazione dei target resta vaga, il motore può sempre “trovare dopo” un target coerente con la traiettoria osservata. Per chiudere davvero la specifica, il catalogo dei target deve essere prodotto da un operatore deterministico e causale.

Sia quindi $\mathcal{T}$ un operatore che, dato lo stato osservabile $\mathcal{F}_t$ e l'indice bersaglio i , produce un insieme finito di target candidati:

$$\mathcal{C}_{i,t}^{target} = \mathcal{T}(\mathcal{F}_t, i) = \{m_1, m_2, \dots, m_M\}.$$

Ogni candidato m_j è a sua volta una tupla che contiene almeno:

$$m_j = (\text{class}, [g_L, g_H], h^{ref}, q^{pers}, q^{liq}, q^{idx}, \text{meta}).$$

In questa notazione:

- **class** identifica la famiglia strutturale del target;
- $[g_L, g_H]$ è la target zone candidata;
- h^{ref} è l'orizzonte o il livello gerarchico di riferimento;
- q^{pers} misura la persistenza della struttura nel tempo;
- q^{liq} misura la compatibilità con la liquidità eseguibile;
- q^{idx} misura la coerenza cross-index;
- **meta** raccoglie attributi ausiliari (sessione, sorgente, score parziali, versione dell'operatore).

11.2 Famiglie ammesse del catalogo strutturale

Il baseline integrato ammette cinque famiglie di target, tutte definite **ex ante** su operatori causali:

1. **estremi e balance boundaries di sessione**: massimi/minimi e bordi di aree di equilibrio delle sessioni precedenti, purché persistenti oltre una soglia di rumore locale;
2. **cluster di swing pivot multi-orizzonte**: zone ricavate dall'aggregazione di pivot consistenti su più orizzonti e non da un singolo massimo/minimo isolato;
3. **nodi di volume e discontinuità strutturali**: high-volume nodes, low-volume nodes, gap o fair-value dislocations persistenti laddove disponibili nel feed di ricerca;
4. **bordi di strutture direzionali incomplete**: livelli di completamento di pattern strutturali già osservabili, purché definiti senza guardare oltre l'istante di emissione;
5. **proiezioni cross-index coerenti**: livelli che risultano strutturalmente supportati da movimenti sincronizzati o divergenze stabili degli indici di contesto.

Il punto non è aumentare arbitrariamente il numero di target: il punto è consentire al motore di lavorare su un insieme finito e replicabile di alternative plausibili.

11.3 Operatore di scoring strutturale

Per ogni target candidato m , il baseline definisce uno score strutturale composito:

$$S_{struct}(t, m, s) = \omega_1 S_{pers}(t, m) + \omega_2 S_{multiH}(t, m) + \omega_3 S_{liq}(t, m, s) + \omega_4 S_{idx}(t, m) - \omega_5 P_{crowd}(t, m) - \omega_6 P_{frag}(t, m).$$

Dove:

- S_{pers} misura quanto a lungo la struttura è rimasta rilevante senza essere neutralizzata;
- S_{multiH} misura il consenso tra orizzonti differenti;
- S_{liq} penalizza target che richiedono attraversamenti sistematicamente incompatibili con spread e profondità;
- S_{idx} misura l'allineamento con il contesto multi-indice;
- P_{crowd} penalizza livelli troppo affollati o compressi tra molte strutture quasi coincidenti;
- P_{frag} penalizza strutture fragili, ossia definite da pochi eventi o da segmenti isolati.

Il motore può pubblicare un segnale solo su target per cui S_{struct} supera una soglia minima. Non basta dunque che un livello sia “visibile”; deve essere anche **strutturalmente preferibile** rispetto alle alternative presenti nel catalogo.

11.4 Regola di selezione del target nel caso di più candidati validi

Se più target candidati superano i filtri di fattibilità, il sistema non deve pubblicare segnali duplicati o semanticamente conflittuali sul medesimo indice e lato nello stesso istante. Il baseline fissa perciò una politica di selezione lessicografica:

1. massimizzare la probabilità end-to-end Π^{E2E} ;
2. a parità, massimizzare S_{struct} ;

3. a parità, minimizzare il rischio di coda $CVaR$ condizionato all'esecuzione;
4. a parità, privilegiare il target con minore distanza strutturalmente coerente, per ridurre la necessità di invalidazioni eccessivamente ampie;
5. a parità residua, usare un tie-break deterministico su `class`, $h^{\sim}\{\text{ref}\}$ e identificatore del candidato.

Questa regola trasforma il catalogo dei target in un vero problema di scelta discreta e impedisce pubblicazioni multiple difficili da auditare.

11.5 Coerenza tra target e geometria della zona

Il baseline integrato impone una logica *target-first*: il target non è conseguenza della zona, ma la zona è costruita come geometria compatibile con un target strutturale già scelto. Ne segue che i quantili di ingresso, la durata e l'invalidazione vengono tutti condizionati su $(\mathcal{F}_t, m, s)$ e non solo su $(\mathcal{F}_t, s)$.

In altre parole, il sistema non stima semplicemente “dove conviene entrare” ma stima “dove conviene entrare **per raggiungere quel target strutturale** senza violare i vincoli del contratto”. Questa è la ragione per cui il catalogo dei target è parte costitutiva, e non ancillare, della specifica.

11.6 Filtri di esclusione del catalogo

Prima ancora del punteggio finale, il documento congela filtri di esclusione obbligatori. Un target candidato viene scartato se si verifica almeno una delle condizioni seguenti:

- distanza strutturale incompatibile con l'orizzonte massimo ammesso dal contratto;
- target già neutralizzato da un percorso di prezzo osservato che lo rende metodologicamente “consumato”;
- target dipendente da informazione non disponibile causalmente all'istante di emissione;
- target che richiederebbe una zona di ingresso semanticamente incoerente con il lato LONG/SHORT;
- target che impone una invalidazione hard talmente ampia da rendere il segnale economicamente non fattibile.

In questo modo il catalogo non è un serbatoio indiscriminato di livelli, ma un insieme già ripulito da alternative metodologicamente non valide.

Eteroschedasticità univariata: famiglie, confronto e ruolo nel motore

12.1 Scomposizione di base

Per ciascun indice i :

$$r_{i,t} = \mu_{i,t} + \varepsilon_{i,t}, \quad \varepsilon_{i,t} = \sigma_{i,t} z_{i,t}.$$

Le innovazioni $z_{i,t}$ possono essere gaussiane, Student-t o skewed, ma la struttura minima richiede che la varianza condizionale sia tempo-variabile.

12.2 GARCH(1,1)

La forma classica:

$$\sigma_{i,t}^2 = \omega_i + \alpha_i \varepsilon_{i,t-1}^2 + \beta_i \sigma_{i,t-1}^2.$$

GARCH è parsimonioso, interpretabile e spesso sufficiente come benchmark. Tuttavia, in mercati indicizzati con shock asimmetrici, gap e clustering di volatilità non lineare, può essere troppo rigido.

12.3 EGARCH(1,1)

Una baseline più robusta in questo contesto è:

$$\log \sigma_{i,t}^2 = \omega_i + \beta_i \log \sigma_{i,t-1}^2 + \alpha_i (|z_{i,t-1}| - \mathbb{E}|z|) + \gamma_i z_{i,t-1} + \delta_i^\top u_{i,t-1}.$$

EGARCH consente di catturare asimmetrie leverage e non richiede vincoli di non negatività diretta sui coefficienti di varianza. Per il problema qui trattato, EGARCH è spesso preferibile come baseline di ricerca.

12.4 GJR-GARCH

GJR-GARCH introduce esplicitamente un termine di soglia per shock negativi/positivi:

$$\sigma_{i,t}^2 = \omega_i + \alpha_i \varepsilon_{i,t-1}^2 + \gamma_i \mathbf{1}_{\{\varepsilon_{i,t-1} < 0\}} \varepsilon_{i,t-1}^2 + \beta_i \sigma_{i,t-1}^2.$$

Nel motore genetico, questa famiglia è utile come alternativa asimmetrica più semplice di EGARCH.

12.5 Realized GARCH

Quando il sistema dispone di misure intraday di volatilità realizzata, è sensato usare un framework tipo Realized GARCH, in cui una misura osservata di volatilità entra come measurement equation. Il vantaggio è duplice:

- la volatilità latente viene ancorata a una proxy più efficiente;
- il layer che genera ampiezza e TTL della zona usa un'informazione più ricca della sola serie close-to-close.

12.6 Conclusione metodologica sul layer univariato

Nel motore genetico, i parametri interni dei modelli univariati non dovrebbero essere geni diretti. I geni devono scegliere la famiglia di volatilità, i regressori ammissibili e le politiche di fallback. I coefficienti veri e propri vanno stimati fold per fold nel protocollo temporale corretto.

Covarianza condizionale multi-indice

13.1 Decomposizione generale

La decomposizione di base è:

$$H_t = D_t R_t D_t,$$

dove $D_t = \text{diag}(\sigma_{1,t}, \dots, \sigma_{N,t})$ e R_t è la matrice di correlazione condizionale dinamica.

Questa forma separa la dinamica univariata della volatilità dalla dinamica della dipendenza cross-index.

13.2 CCC come benchmark

CCC assume $R_t = R$ costante nel tempo. Serve come benchmark parsimonioso e come fallback in caso di instabilità del layer dinamico, ma non è sufficiente quando l'obiettivo è modellare stress sistemico e contagio.

13.3 DCC come baseline parsimoniosa

La forma DCC(1,1) standard:

$$Q_t = (1 - a - b)\bar{Q} + az_{t-1}z_{t-1}^\top + bQ_{t-1},$$

$$R_t = \text{diag}(Q_t)^{-1/2} Q_t \text{diag}(Q_t)^{-1/2},$$

con $a \geq 0$, $b \geq 0$, $a + b < 1$.

DCC è adatto a questo motore perché consente di far crescere il numero di indici di contesto senza far esplodere il numero dei parametri nel layer di correlazione.

13.4 ADCC e correlazioni asimmetriche

In contesti di stress, le correlazioni fra indici non aumentano in modo simmetrico rispetto ai segni degli shock. ADCC introduce termini asimmetrici che rendono il modello più adatto a catturare

fasi di contagio e flight to quality.

13.5 BEKK come opzione PSD-robusta

BEKK parametrizza direttamente H_t in modo da garantire positiva definitezza strutturale. È meno parsimonioso di DCC ma utile come famiglia di confronto e come modello di audit quando DCC appare instabile o produce matrici numericamente problematiche.

13.6 SP-DCC e RSP-DCC come benchmark semiparametrici

Il documento integra inoltre benchmark **semiparametrici** e **regularized semiparametrici** del tipo SP-DCC / RSP-DCC per scenari in cui il layer parametrico tradizionale diventa fragile. Questi benchmark sono particolarmente rilevanti quando:

- $N \geq 12$ indici o il sistema opera su universi cross-index molto correlati;
- il controllo degli autovalori di H_t richiede shrinkage più aggressivo;
- la sensibilità del publish gate o della guardia di regime dipende troppo da una parametrizzazione puramente parametrica.

Il baseline non li promuove a default, ma congela il loro ruolo di benchmark di robustezza e di stress test sul layer di covarianza.

13.7 Rischio sistemico proiettato

Dato un vettore di esposizione u , si definisce lo stress sistemico proiettato:

$$\sigma_t^{sys}(u) = \sqrt{u^\top H_t u}.$$

Questa quantità riassume la componente cross-index del rischio e viene usata direttamente in tre punti del motore:

1. ampiezza della entry zone;
2. distanza di invalidazione;
3. test di persistenza del regime fra emissione e touch.

13.8 Requisiti di stabilità

Il deployment di una specifica non è ammissibile se:

- i residui standardizzati mantengono effetti ARCH residui oltre soglia;
- H_t perde la proprietà PSD;
- la stima numerica produce correlazioni patologiche o strutturalmente instabili.

Il motore quindi non tratta il layer di covarianza come un accessorio descrittivo, ma come un blocco con criteri di qualità minimi verificabili.

Protocollo di stima, distribuzioni delle innovazioni e politiche di fallback

14.1 Mean model e distribuzione delle innovazioni

Per chiudere una delle aperture principali dei sorgenti, il baseline adotta esplicitamente un *mean layer* parsimonioso. Per ogni indice i si assume come forma di riferimento:

$$r_{i,t} = \mu_{i,t} + \varepsilon_{i,t}, \quad \mu_{i,t} = c_i + \phi_i r_{i,t-1} + \psi_i^\top v_{i,t-1},$$

dove $v_{i,t-1}$ raccoglie un numero ridotto di regressori esogeni causalmente osservabili (stato di sessione, bucket temporale, shock realizzati recenti, indicatori di regime). Se nel fold di training il contributo del mean layer non è stabile o non supera criteri minimi di significatività/robustezza, il fallback normativo è un modello zero-mean. La specifica quindi non impone di “prevedere il rendimento medio” a tutti i costi; impone di non confondere tale previsione con la dinamica del rischio.

Le innovazioni standardizzate seguono per default una Student-t standardizzata. La skew-t è ammessa solo quando produce simultaneamente:

- miglioramento stabile di log-verosimiglianza penalizzata;
- riduzione di residui estremi mal calibrati;
- nessun incremento patologico della variabilità dei parametri tra fold.

Questa scelta chiude l’ambiguità sulla distribuzione delle code, che è determinante per CVaR, calibratura e hazard di eventi rari.

14.2 Policy univoca di stima MLE/QMLE

Per evitare che due implementazioni compatibili “a parole” producano bundle frozen diversi per sola differenza di metodo di stima, il documento congela la politica seguente:

- modelli univariati: **MLE Student-t standardizzata** come prima scelta;
- fallback univariato: **Gaussian QMLE** solo in presenza di ragione di fallback persistita;
- modelli DCC/ADCC/cDCC: **two-step QMLE** su residui standardizzati;
- modello BEKK: **QMLE di benchmark**, usato come audit di robustezza e non come rimpiazzo silenzioso della baseline.

Le ragioni di fallback ammesse sono tassative:

1. mancata convergenza entro 250 iterazioni;
2. Hessiana finale non invertibile o non positiva definita;
3. gradi di libertà Student-t stimati $\nu \leq 2.1$;
4. violazione di vincoli strutturali della famiglia;
5. diagnostiche minime fallite dopo un singolo riavvio deterministico dell’ottimizzazione.

Qualunque altro cambio di metodo di stima viene trattato come variazione di specifica e non come dettaglio implementativo.

14.3 Protocollo di stima univariato

Il layer univariato viene stimato fold per fold secondo questa sequenza immutabile:

1. definizione della finestra causale di training del fold;
2. eventuale pulizia robusta di outlier chiaramente spurii di feed, senza alterare eventi di mercato reali;
3. stima del mean layer parsimonioso;
4. stima del modello di volatilità scelto dal gene m^σ ;
5. calcolo dei residui standardizzati $z_{i,t} = \varepsilon_{i,t}/\sigma_{i,t}$;
6. diagnostiche su residui e residui quadratici;
7. eventuale fallback su famiglia più parsimoniosa se la stima fallisce i criteri minimi.

Il fallback è normato e non discrezionale. Se EGARCH non converge, produce parametri instabili o fallisce le diagnostiche, il sistema degrada a GARCH(1,1); se anche GARCH fallisce, il fold o il cromosoma vengono marcati infeasible.

14.4 Diagnostiche minime obbligatorie

Una specifica non è ammissibile se, sui residui standardizzati, non supera almeno i seguenti controlli:

- Ljung–Box sui residui ai lag 10 e 20 con $p\text{-value} > 0.01$;
- ARCH-LM sui quadrati dei residui ai lag 5 e 10 con $p\text{-value} > 0.01$;
- volatilità sempre positiva e varianze diagonali $> 10^{-10}$;
- autovalore minimo di H_t non inferiore a -10^{-8} prima di eventuale repair numerico dichia-

rato;

- persistenza governata: per GARCH/GJR $\alpha + \beta < 0.998$; per EGARCH $|\beta| < 0.998$;
- parametri che collassano o cambiano segno in modo incompatibile con la famiglia scelta.

In forma sintetica, un candidato di volatilità/covarianza è deployable solo se supera simultaneamente test di qualità della stima, qualità numerica e qualità semantica.

14.5 Measurement equation per Realized GARCH

Quando il gene seleziona la famiglia Realized GARCH, il baseline integra una measurement equation del tipo:

$$\log RV_{i,t} = \xi_i + \varphi_i \log \sigma_{i,t}^2 + \tau_i(z_{i,t}) + u_{i,t},$$

dove $RV_{i,t}$ è la realized volatility osservata e $\tau_i(z_{i,t})$ cattura eventuali effetti di leverage o non linearità. Questa estensione viene ammessa solo se la misura realizzata è disponibile in modo coerente nel dataset di ricerca e se il suo utilizzo non rompe la causalità del fold.

14.6 Protocollo di stima del layer DCC/ADCC/BEKK

La covarianza condizionale viene stimata dopo la standardizzazione dei residui. Il baseline DCC segue la canonica decomposizione a due stadi, ma la chiude con regole di stabilizzazione numerica:

- la matrice di lungo periodo $\bar{Q}$ è shrinkata verso una matrice diagonale o verso la correlazione di lungo periodo regolarizzata quando necessario;
- gli autovalori troppo piccoli vengono alzati a una soglia numerica ϵ strettamente positiva per evitare problemi di inversione nella normalizzazione;
- ogni stima viene rifiutata se produce correlazioni non plausibili, parametri fuori dominio o rotture sistematiche della PSD.

Nel caso ADCC, i termini asimmetrici vengono ammessi solo quando il gain di robustezza in fasi di stress compensa l'aumento di complessità. Nel caso BEKK, il baseline lo considera una famiglia di audit e confronto, utile soprattutto come verifica di robustezza quando DCC appare troppo fragile o numericamente instabile.

14.7 Policy di fallback cross-index

Il layer cross-index adotta una cascata deterministica:

1. tentativo di stima della famiglia scelta dal gene (tipicamente DCC);
2. se la stima fallisce ma i residui univariati sono buoni, fallback a CCC;
3. se anche CCC fallisce i requisiti numerici o semantici, il cromosoma è infeasible nel fold.

La ragione di questa cascata è semplice: in un motore che pubblica contratti persistenti, è preferibile una dipendenza cross-index più semplice ma stabile rispetto a una dinamica sofisticata che produce matrici incoerenti.

14.8 PSD enforcement e regola di rifiuto

Per ogni t del fold, deve valere:

$$H_t \succeq 0.$$

Il controllo non è meramente teorico. Il baseline impone:

- test numerico sugli autovalori di H_t ;
- controllo di plausibilità sulle varianze diagonali;
- blocco del deployment del fold se la PSD viene violata oltre tolleranza numerica.

BEKK garantisce positiva definitezza per costruzione; DCC la mantiene se la ricorrenza è ben stimata e la normalizzazione è numericamente stabile; CCC la eredita da varianze positive e matrice di correlazione valida. Il documento assume queste proprietà come vincoli di qualità, non come note a margine.

14.9 Regularizzazione, shrinkage e robustezza ai collassi di regime

Il baseline ammette esplicitamente l'uso di shrinkage e regolarizzazioni leggere, purché:

- siano deterministiche e documentate nel bundle frozen;
- non usino informazione futura;
- non modifichino il core dei segnali già emessi;
- vengano applicate a monte, nella stima del layer rischio, e non come correzione opportunistica a valle.

Questo punto è importante: una regolarizzazione ben definita rende il motore più stabile; una correzione ad hoc introdotta solo quando il live “disturba” il segnale rompe invece la coerenza tra research e runtime.

Generazione della zona, durata e invalidazione

15.1 Costruzione target-first

Per ogni target strutturale candidato m e lato s , il sistema deve stimare almeno i quantili condizionali di distanza all'entry, distanza al target e durata al touch. La costruzione è target-first: il motore non sceglie una zona arbitraria e poi spera che il target sia compatibile; sceglie una geometria coerente con un target strutturale definito.

15.2 Centro della entry zone

Il centro della zona di ingresso è determinato da un quantile condizionale, non da un offset fisso:

$$c_{t,m,s} = P_t - s Q_{\alpha_e}(|\Delta P_e| \mid \mathcal{F}_t, m, s).$$

Questo consente alla distanza di ingresso di adattarsi al regime attuale senza introdurre refresh futuro del contratto.

15.3 Ampiezza della zona

La semilarghezza deve crescere monotonicamente con rischio e frizione:

$$w_{t,m,s} = \text{clip} \left(w_{min}, w_{max}, k_1 \sigma_{i,t} + k_2 \sigma_t^{sys}(u) + k_3 spread_t + k_4 depth_t^{-1} \right).$$

La zona diventa quindi:

$$Z_{e,t,m,s} = [c_{t,m,s} - w_{t,m,s}, c_{t,m,s} + w_{t,m,s}].$$

15.4 Durata di validità

Il tempo massimo non deve essere espresso come numero fisso di barre. Deve derivare da un quantile di durata:

$$t_{\max} = t_0 + Q_{\beta_\tau}(\tau_e \mid \mathcal{F}_{t_0}, m, s).$$

La durata è quindi parte del contenuto probabilistico del segnale, non un parametro manuale esterno.

15.5 Invalidation price

L'invalidazione pre-entry deve stare dal lato opposto alla zona e crescere con il rischio:

$$I_{t,m,s} = c_{t,m,s} - s(\rho_1 w_{t,m,s} + \rho_2 \sigma_{i,t} + \rho_3 \sigma_t^{sys}(u)).$$

15.6 Condizioni minime di emissione

Un candidato segnale può essere pubblicato solo se tutte le seguenti condizioni sono vere:

$$\begin{aligned} \pi_{touch} &\geq \theta_{touch}, & \pi_{hit} &\geq \theta_{hit}, & \mathbb{E}[R_{net} \mid execute, \mathcal{F}_t] &\geq \theta_R, \\ CVaR_\alpha(L \mid execute, \mathcal{F}_t) &\leq \theta_L, & S_{struct} &\geq \theta_S, & S_{liq} &\geq \theta_Q. \end{aligned}$$

Questa clausola fa sì che il motore non pubblichi zone solo perché matematicamente definibili; le pubblica solo se superano un filtro economico e strutturale.

Survival pre-entry: touch, guardie e competing risks

16.1 Perché il touch non è una semplice classificazione binaria

Ridurre il problema a “la zona verrà toccata nella prossima barra?” è concettualmente errato. Un segnale persistente vive per un intervallo e può:

- non essere mai toccato;
- essere toccato ma non diventare eseguibile;
- essere invalidato prima del touch;
- vedere il target raggiunto prima dell’entry;
- diventare eseguibile al touch.

La formulazione corretta è quindi un problema di tempo al primo evento con rischi concorrenti.

16.2 Hazard di raw touch

Su una griglia discreta di eventi $k = 1, \dots, K$, una forma utile è il complementary log-log:

$$h^{touch}(k | x_k) = 1 - \exp\{-\exp(\alpha_k^{touch} + \beta_{touch}^\top x_k)\}.$$

Il vettore x_k include distanza residua dalla zona, volatilità locale, stress sistemico, spread, depth, standardized shocks e stato di regime.

16.3 Guardie pass, soft fail e hard reject

Per rispettare la semantica runtime del modello, il raw touch va fattorizzato ulteriormente:

$$p^{pass}(k | g_k) = \mathbb{P}(\text{tutte le guardie passano} | \text{raw touch in } k, g_k),$$

$$p^{hard}(k | g_k) = \mathbb{P}(\text{hard reject} | \text{raw touch in } k, g_k),$$

$$p^{soft}(k \mid g_k) = 1 - p^{pass}(k \mid g_k) - p^{hard}(k \mid g_k).$$

Ne segue:

$$h^{exe}(k) = h^{touch}(k) p^{pass}(k),$$

$$h^{rej}(k) = h^{touch}(k) p^{hard}(k).$$

Le soft failure non tolgono il segnale dal risk set: registrano il touch e lasciano il contratto vivo.

16.4 Altri rischi concorrenti pre-entry

Si modellano almeno:

$$h^{inv}(k) = \mathbb{P}(\text{price or vol invalidation in } k \mid \text{alive}),$$

$$h^{miss}(k) = \mathbb{P}(\text{target before entry in } k \mid \text{alive}).$$

16.5 Sopravvivenza del segnale pending

La survival function del segnale pending è:

$$S(k) = \prod_{u=1}^k (1 - h^{exe}(u) - h^{rej}(u) - h^{inv}(u) - h^{miss}(u)).$$

La probabilità di diventare effettivamente eseguibile entro expiry è:

$$\Pi^{exe}(K) = \sum_{k=1}^K S(k-1) h^{exe}(k).$$

Questa quantità è molto più fedele al comportamento reale del motore rispetto a un touch-rate grezzo.

Survival post-entry: target, stop e timeout

17.1 Necessità di un secondo blocco

Il fatto che un segnale diventi executable non basta. Occorre stimare se, dopo fill, il target strutturale possa essere raggiunto prima di stop, timeout o decadimento.

17.2 Hazard post-entry

Definiamo, per u tempo dal fill:

$$h^{tar}(u \mid k, y_u), \quad h^{stop}(u \mid k, y_u), \quad h^{tout}(u \mid k, y_u).$$

Qui k è il tempo al quale è avvenuta l'esecuzione e y_u sono covariate post-entry.

La probabilità di target dato fill al tempo k è:

$$\Pi^{tar|exe}(H - k) = \sum_{u=1}^{H-k} S^{post}(u - 1 \mid k) h^{tar}(u \mid k).$$

17.3 Probabilità end-to-end

La quantità operativamente più importante è:

$$\Pi^{E2E}(K, H) = \sum_{k=1}^K S(k - 1) h^{exe}(k) \Pi^{tar|exe}(H - k).$$

Questa misura riassume l'intera catena:

emissione -> persistenza -> raw touch -> pass delle guardie -> fill -> target.

È la grandezza che dovrebbe guidare tanto la fitness del motore quanto le soglie minime di pubblicazione.

Guardie di regime e persistenza eteroschedastica

18.1 Eteroschedasticità come regola di permanenza

Una delle acquisizioni più importanti del modello è che la volatilità condizionale e la covarianza dinamica non servono solo a dimensionare la zona al tempo di emissione; servono anche a decidere se il segnale rimane vivo.

18.2 Volatility regime break

Una guardia hard naturale è:

$$\frac{\sigma_{i,\tau}}{\sigma_{i,t_0}} > \theta_\sigma \quad \vee \quad \frac{\lambda_{\max}(H_\tau)}{\lambda_{\max}(H_{t_0})} > \theta_H \quad \vee \quad spread_\tau > \theta_{spr}^{hard}.$$

Se una qualunque di queste condizioni scatta prima dell'esecuzione, allora:

$$state = INVALIDATED.$$

18.3 Guardie soft

Le guardie soft possono includere:

- profondità insufficiente ma non critica;
- microstructure noise elevato;
- spike di volatilità sotto soglia hard ma sopra soglia soft;
- standardized residuals estremi;
- sessione o bucket temporale non ottimale.

Le soft guard non chiudono il segnale, ma riducono p^{pass} al successivo touch.

Architettura del motore genetico

19.1 Cosa codifica il cromosoma

Un cromosoma appropriato non include coefficienti econometrici stimati fold per fold; include invece le scelte di architettura. Una forma generale è:

$$\chi = (\mathcal{J}, m^\mu, m^\sigma, m^R, \alpha_e, \beta_\tau, k_1, k_2, k_3, k_4, \rho_1, \rho_2, \rho_3, \theta_{touch}, \theta_{E2E}, \theta_\sigma, \theta_H, \theta_{spr}, \theta_{depth}, \mathcal{G}_{soft}, \mathcal{G}_{hard}, \mathcal{P}_{conflict}).$$

Dove:

- $\mathcal{J}$ è il subset di feature;
- m^σ sceglie fra GARCH, EGARCH, GJR, Realized GARCH o equivalenti;
- m^R sceglie fra CCC, DCC, ADCC, BEKK o fallback;
- α_e e β_τ controllano quantili di entry e durata;
- k controlla l'ampiezza;
- ρ l'invalidazione;
- θ le soglie di fattibilità e di guardia.

19.2 Inizializzazione della popolazione

Per passare dalla SRS all'implementazione replicabile, questa monografia raccomanda:

- sampling iniziale con vincoli di feasibility già applicati;
- copertura obbligatoria di tutte le famiglie modello ammesse;
- seme di baseline per almeno una soluzione parsimoniosa e una soluzione asimmetrica.

19.3 Operatori per geni misti

Poiché il cromosoma contiene geni discreti e continui, gli operatori dovrebbero essere differenziati:

- crossover uniforme o a blocchi per geni discreti;
- simulated binary crossover o blend crossover per geni continui;
- mutazione categorica per scelte di famiglia modello;

- mutazione gaussiana o polynomial mutation per parametri continui.

19.4 Fitness multi-obiettivo

Una scalarizzazione unica è prematura e pericolosa. La forma consigliata è una ricerca Pareto-constrained in stile NSGA-II. Gli obiettivi minimi possono essere espressi come:

$$\begin{aligned}
 f_1(\chi) &= -DSR_{OOS}(\chi), \\
 f_2(\chi) &= CVaR_{0.95}(L_\chi), \\
 f_3(\chi) &= Brier_\chi^{exe} + Brier_\chi^{tar}, \\
 f_4(\chi) &= \text{Var}_{r \in \mathcal{R}} E[R_{net} \mid \chi, r] + \text{Var}_{i \in \mathcal{J}} E[R_{net} \mid \chi, i], \\
 f_5(\chi) &= \lambda_p N_{param} + \lambda_f N_{feat} + \lambda_g N_{guards}.
 \end{aligned}$$

19.5 Vincoli

Le principali constraint dovrebbero includere:

$$\begin{aligned}
 \Pi_{OOS}^{exe} &\geq \pi_{exe}, & \Pi_{OOS}^{E2E} &\geq \pi_{E2E}, \\
 \widehat{InvalidRate}_{OOS} &\leq \bar{v}, & \widehat{PBO} &\leq \theta_{PBO}, & H_t &\succeq 0 \quad \forall t.
 \end{aligned}$$

19.6 Regola di dominance vincolata

Si preferisce χ_a a χ_b se:

1. χ_a è feasible e χ_b no;
2. entrambe sono feasible e χ_a Pareto-domina χ_b ;
3. entrambe sono infeasible ma χ_a ha violazione totale minore.

Questa regola impedisce al motore di privilegiare soluzioni con performance apparente alta ma semantica o stabilità inaccettabili.

19.7 Criteri di arresto

Per rendere replicabile il processo evolutivo, si raccomanda di fissare:

- numero massimo di generazioni;
- criterio di stabilizzazione dell'hypervolume o della frontiera Pareto;
- early stopping se la frontiera interna migliora ma il test outer degrada;
- limite massimo di complessità architetturale.

Parametrizzazione concreta del motore genetico, domini dei geni e penalizzazione di complessità

20.1 Domini dei geni

Per chiudere il motore genetico in modo realmente implementabile, il documento fissa anche il dominio ammissibile dei geni principali. La tabella seguente va letta come contratto del baseline, non come semplice esempio.

Gene	Tipo	Dominio baseline	Note
$\mathcal{J}$	discreto strutturato	sottoinsieme di feature ammesse, con vincolo minimo/massimo	nessuna feature fuori catalogo
m^μ	discreto	{zero-mean, ARX(1)}	parsimonia obbligatoria
m^σ	discreto	{EGARCH, GARCH, GJR, Realized GARCH}	baseline EGARCH
m^R	discreto	{DCC, CCC, ADCC, BEKK}	baseline DCC
α_e	continuo	quantile ingresso in [0.50, 0.95]	quantili troppo estremi sono vietati
β_τ	continuo	quantile durata in [0.50, 0.95]	evita TTL troppo corti o troppo lunghi
k_1, k_2, k_3, k_4	continui	non negativi, con bound superiori finiti	garantiscono monotonia dell'ampiezza

Gene	Tipo	Dominio baseline	Note
ρ_1, ρ_2, ρ_3	continui	non negativi, con bound superiori finiti	invalidazione sempre sul lato coerente
$\theta_{touch}, \theta_{E2E}$	continui	soglie minime probabilistiche	gating di emissione
θ_σ, θ_H	continui	soglie di regime break > 1	invalidazione hard di regime
$\theta_{spr}, \theta_{depth}$	continui	soglie microstrutturali	hard/soft guard
$\mathcal{G}_{soft}, \mathcal{G}_{hard}$	discreti	sottoinsiemi di guardie ammesse	nessuna guard non dichiarata
$\mathcal{P}_{conflict}$	discreto	politica lessicografica / single-best / single-per-side	baseline single-best per indice/lato

20.2 Inizializzazione feasible-first

La popolazione iniziale non viene campionata in modo ingenuo. Il baseline congela una procedura **feasible-first**:

1. si campionano prima i geni discreti assicurando copertura di tutte le famiglie ammesse;
2. si campionano poi i geni continui tramite Latin Hypercube o campionamento stratificato nei rispettivi domini;
3. ogni cromosoma viene proiettato nello spazio ammissibile se viola vincoli di base (ad esempio coefficienti negativi per ampiezza o invalidazione);
4. i cromosomi non riparabili vengono rigenerati, non “tenuti comunque”.

Questa scelta riduce la dispersione inutile della ricerca in regioni palesemente inammissibili e accelera la comparazione di architetture realmente plausibili.

20.3 Dominance vincolata e archivio elitista

La ricerca multi-obiettivo usa NSGA-II con dominance vincolata. Il criterio di confronto tra due cromosomi χ_a e χ_b è:

- precedenza assoluta alla fattibilità;
- a parità di fattibilità, dominanza Pareto sugli obiettivi;
- in caso di non-dominanza, crowding distance per preservare diversità;
- se entrambi infeasible, minore violazione totale.

L’archivio elitista conserva il fronte non dominato del livello outer-valid, non semplicemente il migliore sul training interno. In questo modo l’elitismo non preserva il “miglior backtest” ma la migliore frontiera compatibile con il protocollo OOS.

20.4 Penalizzazione di complessità

Il baseline fissa esplicitamente una penalità di complessità:

$$P_{complex}(\chi) = \lambda_p N_{param}(\chi) + \lambda_f N_{feat}(\chi) + \lambda_g N_{guards}(\chi) + \lambda_m N_{models}(\chi),$$

dove il termine finale penalizza la combinazione contemporanea di troppi layer modellistici complessi. Non basta quindi un numero basso di feature per essere “semplici”: una soluzione che usa skew-t, Realized GARCH, ADCC e molte guardie runtime è comunque più complessa di una soluzione con EGARCH + DCC + guard set parsimonioso.

20.5 Criteri di arresto integrati

I criteri di stop non sono lasciati al gusto dell’implementatore. Il baseline usa un arresto composito e **numericamente congelato**:

1. limite massimo di generazioni $G_{max} = 150$;
2. plateau dell’hypervolume su obiettivi normalizzati in $[0, 1]^m$, reference point $r = (1.05, \dots, 1.05)$, $\epsilon_{HV} = 10^{-4}$ e $G_{stall} = 15$;
3. early stop se il DSR mediano di outer-valid peggiora per 5 generazioni consecutive mentre l’hypervolume inner continua a migliorare;
4. arresto forzato se oltre l’80% della popolazione è infeasible per 10 generazioni consecutive.

La condizione di plateau è:

$$\max_{j=g-G_{stall}+1, \dots, g} |HV_j - HV_{j-1}| \leq \epsilon_{HV}.$$

20.6 Selezione finale del bundle frozen

La selezione finale del bundle da congelare non coincide automaticamente con il primo elemento del fronte di Pareto. Il documento congela una procedura di *bundle selection* a due stadi:

- primo stadio: identificazione del fronte feasible su outer-valid/test;
- secondo stadio: scelta della soluzione da congelare tramite ordinamento lessicografico su GO/CONDITIONAL/NO-GO, DSR, PBO, robustezza cross-regime, complessità.

Questo evita l’errore concettuale di trattare il fronte di Pareto come se producesse da solo una “soluzione ufficiale”. La soluzione ufficiale emerge invece da un atto finale di governance quantitativa, ma sempre su regole congelate.

Macchina a stati runtime

21.1 Stati minimi

Gli stati minimi del lifecycle segnale/ordine sono:

- PENDING;
- EXECUTABLE;
- ORDER_WORKING;
- EXECUTED;
- EXPIRED;
- INVALIDATED;
- MISSED_TARGET;
- REJECTED_TOUCH;
- ORDER_REJECTED;
- CANCELLED_UNFILLED.

Il lifecycle della posizione oltre il fill è un sottosistema distinto e non va confuso con il lifecycle del contratto di segnale.

21.2 Precedenza degli eventi

A parità di timestamp, la precedenza corretta è:

1. expiry;
2. invalidation di prezzo o di regime;
3. target before entry;
4. touch;
5. valutazione delle runtime guards;
6. pretrade gate;
7. azione ordine.

Questa precedenza è essenziale per rendere deterministico il replay e coerente la validazione con il runtime reale.

21.3 Transizioni principali

Dal PENDING:

- se il tempo non è ancora valido, il segnale resta pending;
- se scade, va in expired;
- se si invalida per prezzo o regime, va in invalidated;
- se il target è raggiunto prima dell'entry, va in missed target;
- se raw touch avviene con sole soft fail, incrementa touch count e resta pending;
- se raw touch avviene con hard fail, va in rejected touch o invalidated;
- se raw touch avviene con tutte le guardie valide, va in executable.

Da EXECUTABLE:

- se il pretrade gate fallisce, va in rejected touch;
- se l'ordine è accettato, va in order working;
- se l'ordine è rifiutato dalla venue, va in order rejected.

Da ORDER_WORKING:

- se arriva il fill, va in executed;
- se scade o si attiva una cancel rule senza fill, va in cancelled unfilled.

21.4 Proprietà desiderate della macchina

La state machine deve essere:

- deterministica;
- idempotente al replay;
- resistente al restart;
- compatibile con logging evento per evento;
- indipendente dal fatto che il modello sia stato retrainato dopo l'emissione.

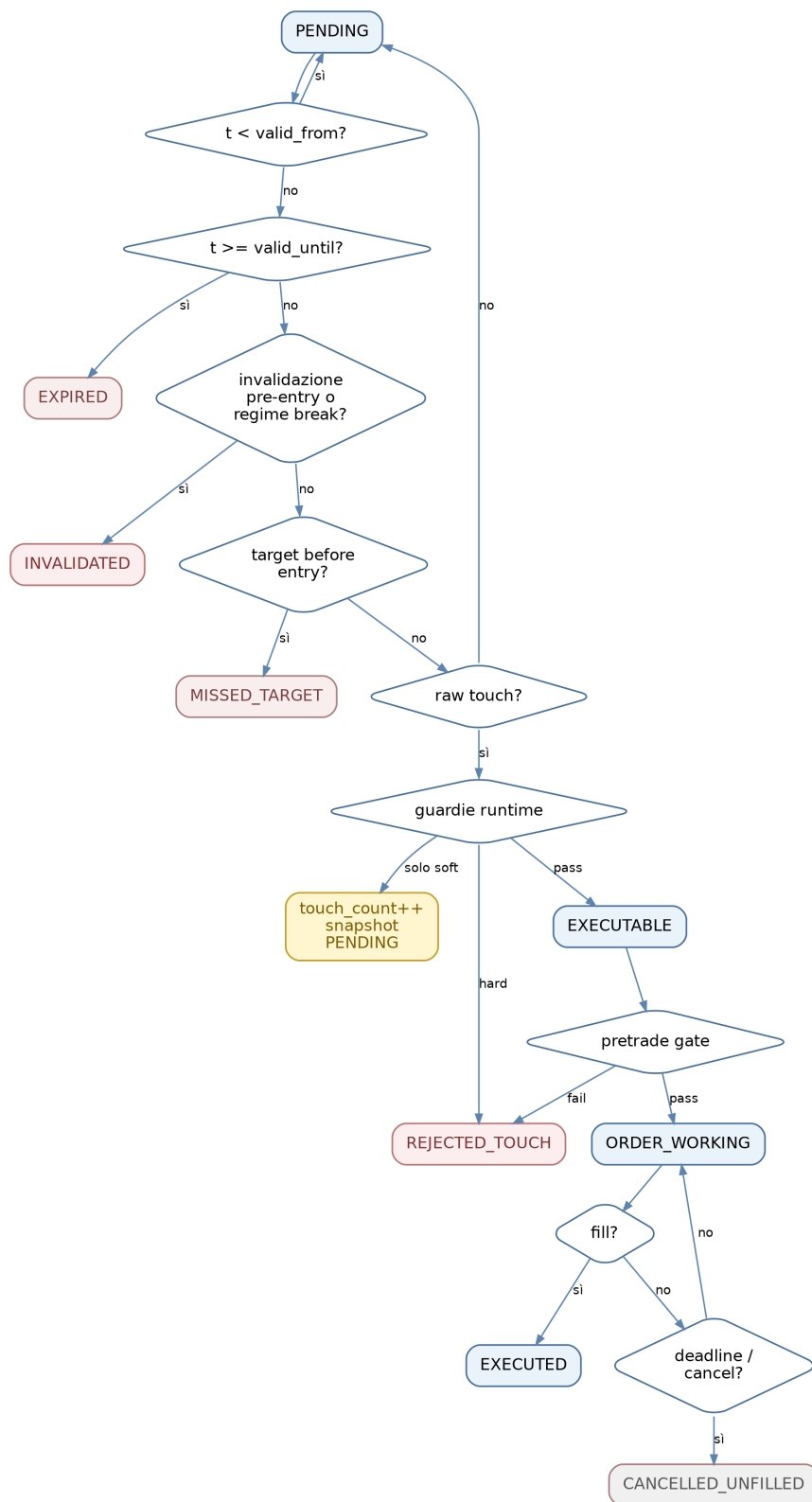


Figura 21.1: Figura 2 — Macchina a stati runtime del contratto di segnale. Anche in questo caso la fonte logica canonica è conservata in Mermaid e la figura integrata nel PDF è la versione frozen del bundle documentale.

Eventi speciali, finestre esecutive e casi limite della macchina a stati

22.1 Perché servono casi limite formalizzati

Una state machine apparentemente completa può restare ambigua proprio nei casi che, in produzione, generano i problemi peggiori: touch al limite della validità, ordini non inviati in tempo, fill e cancel con timestamp identico, restart durante `ORDER_WORKING`, ordine parzialmente eseguito. Il baseline integrato non lascia questi punti come decisioni locali del codice: li congela esplicitamente.

22.2 Stato terminale `EXECUTION_WINDOW_MISSED`

Il documento introduce come stato terminale esplicito `EXECUTION_WINDOW_MISSED`. Questo stato si applica quando il segnale è diventato `EXECUTABLE`, ma l'ordine non è stato inviato entro una finestra esecutiva massima Δ_{exe} definita nel bundle di runtime.

Formalmente, se τ_{exe} è il tempo in cui il segnale entra in stato `EXECUTABLE`, allora si definisce una scadenza esecutiva:

$$\tau_{deadline}^{exe} = \tau_{exe} + \Delta_{exe}.$$

Se nessun evento di submit ordine valido avviene entro $\tau_{deadline}^{exe}$, il segnale termina in `EXECUTION_WINDOW_MISSED`. Questa scelta separa in modo netto due situazioni diverse:

- un segnale mai diventato eseguibile;
- un segnale diventato eseguibile ma non tradotto in ordine entro il tempo ammesso.

22.3 `cancel_on_expiry` in `ORDER_WORKING`

Nel baseline integrato, `ORDER_WORKING` non si chiude immediatamente alla semplice occorrenza di `valid_until` se è attiva la policy `cancel_on_expiry`. La regola corretta è:

1. al raggiungimento di `valid_until`, se l'ordine è ancora `working`, il sistema alza il flag `cancel_requested = true`;
2. lo stato resta `ORDER_WORKING` finché non arriva un evento di `cancel` confermato o di `fill`;
3. se arriva il `fill` prima della conferma di `cancel`, prevale il `fill`;
4. se arriva la conferma di `cancel` senza `fill`, lo stato passa a `CANCELLED_UNFILLED`.

Questa scelta rende il runtime coerente con i mercati reali, dove tra richiesta di `cancel` e conferma definitiva può esistere una latenza non trascurabile.

22.4 Precedenza `fill > cancel` a timestamp identico

A parità di timestamp, il baseline impone la precedenza:

$$fill \succ cancel_confirm.$$

La ragione non è solo prudenziale: è microstrutturalmente coerente con il fatto che un ordine può essere eseguito in venue prima che la cancellazione venga riconosciuta o propagata a valle. Questa regola elimina l'ambiguità tipica dei replay in cui lo stesso ordine appare sia cancellato sia fillato nella stessa unità temporale.

22.5 `unsupported_event` e robustezza del dispatcher

Il dispatcher runtime deve essere totale sul dominio degli eventi ammessi e rifiutare in modo esplicito ogni evento non supportato, marcandolo con `reason_code = UNSUPPORTED_EVENT`. Il rifiuto non può produrre transizioni silenziose né side effects nascosti sul segnale. In termini di audit, un evento sconosciuto deve comparire nel ledger come rifiutato, non come assorbito tacitamente.

22.6 Partial fill come boundary esplicito

Il lifecycle del segnale/ordine viene chiuso fino a `EXECUTED`, `ORDER_REJECTED`, `CANCELLED_UNFILLED` e `EXECUTION_WINDOW_MISSED`. La gestione fine dei partial fill viene trattata come submacchina della posizione e dunque come boundary del presente documento. Tuttavia la specifica congela due obblighi:

- il ledger deve poter registrare quantità fillata, quantità residua e timestamp di ogni partial fill;
- la state machine del segnale non deve essere resa incoerente da partial fill successivi.

Il modo più semplice di rispettare il boundary è trattare il primo partial fill come passaggio del segnale in `EXECUTED`, spostando la gestione quantitativa della posizione in un sottosistema distinto.

22.7 Restart e idempotenza degli ordini

Il runtime deve essere restart-safe. Questo significa che, dopo un riavvio:

- il sistema ricostruisce lo stato del segnale dal ledger e dal bundle frozen;
- non vengono generati nuovi ordini per un `signal_id` che ne abbia già generato uno;
- eventuali eventi già consumati non vengono applicati due volte;
- il core hash del segnale resta invariato.

In termini pratici, l'idempotenza degli ordini è un requisito forte quanto la correttezza della matematica del layer quant: un motore impeccabile dal punto di vista econometrico ma non idempotente a runtime non è un motore deployable.

Validazione OOS e controlli anti-overfitting

23.1 Split temporali ordinati

Qualunque training, model selection e fitness evaluation deve rispettare la cronologia. Non è ammessa validazione randomizzata. L'outer loop corretto è walk-forward o expanding window time-ordered.

23.2 Purge ed embargo

Quando le feature o le label usano finestre sovrapposte, il baseline non si limita a prescrivere purge ed embargo in astratto: ne congela la parametrizzazione. Sia H_{label}^{max} il massimo orizzonte temporale con cui una feature, una label o un evento può contaminare causalmente un'altra osservazione del fold. Allora la specifica impone:

$$L_{purge} = H_{label}^{max}, \quad L_{emb} = \max \left(\lceil 0.01 N_{fold} \rceil, \lceil 0.25 H_{label}^{max} \rceil \right).$$

Qui N_{fold} è la numerosità del fold nella stessa unità temporale canonica (barre o bucket event-driven). Nessun report può essere qualificato come baseline se purge o embargo risultano disattivati.

23.3 Nested walk-forward

Il framework raccomandato è nested:

- outer loop come verità OOS per la decisione di deploy;
- inner loop come layer di ricerca per comparare architetture, tarare soglie, stimare robustezza.

Questa separazione riduce il rischio che i criteri di scelta del motore genetico usino indirettamente informazione del test finale.

23.4 Bootstrap dipendente e inferenza robusta

Poiché i rendimenti e le metriche di strategia sono eteroschedastici e autocorrelati, le stime d'incertezza devono essere robuste. Il baseline congela lo stationary bootstrap con:

$$\bar{\ell} = \max\left(10, \lceil 1.5 H_{label}^{max} \rceil\right), \quad p_{SB} = 1/\bar{\ell}, \quad B = 2000.$$

Il motore usa quindi:

- White/HAC/Newey–West per confronti lineari semplici;
- stationary bootstrap con lunghezza media dei blocchi $\bar{\ell}$ e $B = 2000$ replicazioni come default;
- reporting di intervalli al 95% per metriche chiave, con eventuali varianti BCa classificate come `research-only`.

23.5 DSR e PBO

Un motore genetico esplora molte configurazioni. Di conseguenza, il rischio di selection bias è intrinseco. Per questo sono raccomandati due filtri:

- **Deflated Sharpe Ratio** per correggere l'apparente Sharpe tenendo conto di non normalità, multiple prove e selezione;
- **Probability of Backtest Overfitting** via CSCV/CPCV per stimare quanto spesso la soluzione scelta nel training perda ranking nel test.

Questi strumenti non sostituiscono il walk-forward: lo rendono più credibile.

23.6 White Reality Check e Hansen SPA

Quando si confrontano molti candidati o benchmark, è opportuno aggiungere test multipli come White Reality Check o Hansen SPA, trattandoli però come layer di ricerca, non come unica condizione di deploy.

Protocollo di validazione concreto: DSR, PBO, tassonomia dei regimi e gate decisionali

24.1 Topologia del nested walk-forward

Il baseline integrato adotta una topologia nested con tre livelli temporalmente ordinati:

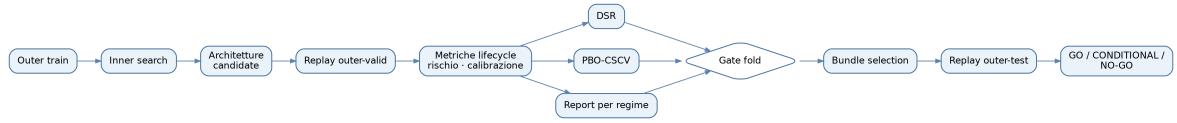


Figura 24.1: Figura 3 — Pipeline di validazione nested walk-forward con side-car DSR/PBO e report per regime. La figura esplicita la separazione tra layer di ricerca e verità OOS del fold.

1. **outer train:** usato per stimare famiglie e parametri dei candidati;
2. **outer valid:** usato per la ricerca guidata e la selezione del fronte feasible;
3. **outer test:** usato come verità OOS finale del fold.

All'interno dell'outer train opera un inner loop temporale che serve per scegliere soglie, quantili e iperparametri senza contaminare outer valid e outer test. Il punto essenziale è che il fold di test non viene mai toccato né indirettamente né tramite tuning guidato.

24.2 Purge, embargo e fit dei regimi

Il protocollo impone purge ed embargo con la parametrizzazione congelata del capitolo hard-locked:

$$L_{purge} = H_{label}^{max}, \quad L_{emb} = \max \left(\lceil 0.01 N_{fold} \rceil, \lceil 0.25 H_{label}^{max} \rceil \right).$$

Inoltre tutti i cutpoint di regime vengono fit **solo** sul train del fold. Sia R_t una metrica di regime, ad esempio la volatilità locale, il rischio sistemico proiettato o uno score combinato. I cutpoint

vengono fissati come quantili del train:

$$(c_1, c_2, \dots, c_K) = \mathcal{Q}_{train}(R_t).$$

Una volta fissati, i medesimi cutpoint vengono applicati a valid e test. Questo impedisce che la tassonomia dei regimi venga “adattata” ex post alla porzione che si vuole giudicare.

24.3 PSR e DSR

Per il controllo di selection bias, il baseline usa il Probabilistic Sharpe Ratio come ingrediente del Deflated Sharpe Ratio. In forma concettuale:

$$PSR = \Phi \left(\frac{(\widehat{SR} - SR^*)\sqrt{T-1}}{\sqrt{1 - \hat{\gamma}_3 \widehat{SR} + \frac{\hat{\gamma}_4 - 1}{4} \widehat{SR}^2}} \right),$$

dove $\widehat{SR}$ è lo Sharpe empirico, SR^* il benchmark di riferimento, T la numerosità effettiva, $\hat{\gamma}_3$ la skewness e $\hat{\gamma}_4$ la kurtosis empirica.

Il DSR sostituisce SR^* con una soglia deflazionata che tiene conto del numero effettivo di prove e della distribuzione delle performance candidate. La specifica non demanda questo passaggio a un commento di ricerca: lo rende parte del gate di selezione del bundle frozen.

24.4 Numero effettivo di prove

Poiché il GA non produce prove indipendenti, il numero di prove da usare nel DSR non coincide banalmente con il numero totale di cromosomi valutati. La versione hard-locked congela la seguente definizione conservativa:

$$N_{eff} = \min \left\{ N_{eval}, N_{front} + \lceil \sqrt{N_{eval}} \rceil \right\},$$

dove N_{eval} è il numero totale di valutazioni outer-valid del fold e N_{front} è la cardinalità del fronte feasible non dominato prima del bundle selection. Questa scelta impedisce di calcolare il DSR fingendo indipendenza totale tra tutte le soluzioni esplorate e fornisce una baseline riproducibile anche in assenza di stime più sofisticate della dipendenza.

24.5 PBO tramite CSCV/CPCV

Il controllo di overfitting usa la Probability of Backtest Overfitting calcolata con schemi CSCV o CPCV. Il principio è il seguente:

1. si generano molte partizioni combinatorie train/test del campione di ricerca;
2. in ogni partizione si seleziona la migliore soluzione sul sotto-campione di training;
3. si osserva il ranking della stessa soluzione sul sotto-campione complementare di test;
4. si stima la probabilità che il ranking crolli sistematicamente fuori campione.

Il baseline considera PBO una misura di *fragilità della selezione*, non semplicemente di performance cattiva. Una strategia può avere buon ritorno medio ma PBO troppo alto, e quindi essere comunque non candidabile al live controllato.

24.6 White Reality Check e Hansen SPA come layer research-only

White Reality Check e Hansen SPA vengono mantenuti come strumenti facoltativi di approfondimento nel layer di ricerca. Essi non sostituiscono DSR e PBO, ma possono rafforzare l'evidenza quando si confrontano molti benchmark o molte famiglie architetture. La specifica li classifica esplicitamente come strumenti **research-only** per evitare che diventino l'unico criterio di deploy decision.

24.7 Gate di fold e decisione globale

Il documento congela una tassonomia di esito a due livelli.

24.7.1 Gate di fold

Ogni fold produce uno fra tre esiti:

- **PASS**: supera i minimi su Π^{exe} , Π^{E2E} , DSR, PBO, PSD, stabilità per regime e qualità del replay;
- **CONDITIONAL**: supera i requisiti minimi di sicurezza ma presenta uno o più warning non terminali (ad esempio DSR marginale o instabilità lieve in un regime raro);
- **FAIL**: viola almeno un vincolo hard.

24.7.2 Decisione globale

Aggregando i fold, il bundle frozen ottiene uno fra tre esiti:

- **GO**: nessun fold FAIL, DSR e PBO globali entro soglia, assenza di collassi sistematici per regime;
- **CONDITIONAL**: pochi warning concentrati e spiegabili, nessun collasso hard, bundle congelabile solo per run controllato;
- **NO_GO**: uno o più fallimenti hard o eccessiva fragilità strutturale.

Questa gerarchia evita di comprimere tutto in una media numerica. Un motore con media buona ma un collasso ricorrente in un regime di stress non è GO.

24.8 Contratto minimo dei report

Ogni fold deve produrre almeno quattro artefatti di report:

1. **replay report**: eventi, transizioni, ragioni terminali, ledgers;
2. **performance report**: distribuzioni di ritorno, CVaR, drawdown, MAE/MFE;
3. **calibration report**: Brier, reliability, calibration bins per π_{touch} , π_{exe} , π_{tar} ;

4. **regime report:** metriche disaggregate per volatilità, correlazione/stress sistemico, liquidità, sessione, famiglia modello.

Senza questo quartetto minimo la validazione non è considerata completa, anche se esiste una singola metrica aggregata apparentemente buona.

Metriche operative e reporting per regime

25.1 Metriche lifecycle

Le metriche devono essere calcolate sul replay della state machine, non su euristiche bar-based.

Le principali includono:

- emission count;
- raw touch rate;
- executable rate;
- hard reject rate;
- invalidation rate;
- missed target rate;
- executed-to-target hit rate;
- order reject rate;
- cancelled unfilled rate.

25.2 Metriche economiche e di rischio

Sul sottoinsieme eseguito vanno riportati almeno:

- expected net return;
- distribuzione dei ritorni netti;
- CVaR;
- max drawdown;
- ulcer index;
- MAE/MFE aggregati;
- costo di opportunità delle zone mai toccate.

25.3 Calibrazione probabilistica

Se il motore produce π_{touch} , π_{exe} , π_{tar} , allora la calibrazione è critica. Si raccomanda:

- Brier score;
- reliability curve;
- analisi dei quantile bins;
- expected calibration error su bucket rilevanti.

25.4 Tassonomia dei regimi

Il reporting OOS deve essere disaggregato almeno per:

- regime di volatilità;
- bucket di correlazione/covarianza cross-index;
- fascia di liquidità e spread;
- sessione o sottoperiodo intraday, se rilevante;
- famiglia modello selezionata.

25.5 Esperimenti di robustezza

Si raccomanda di includere:

- shock test sulle correlazioni, confrontando DCC contro ADCC;
- ablation del vincolo no-refresh in ambiente solo research, per misurare quanto performance apparente derivi da mutazioni ex post del contratto;
- analisi di stabilità parametrica delle soglie fra fold;
- valutazione separata in fasi di mercato calme, turbolente e di contagio.

Reproducibilità, bundle frozen e industrializzazione

26.1 Problema generale

Una specifica metodologica può essere formalmente corretta e tuttavia non diventare mai un motore operativo se la catena di inferenza non è congelabile e ripristinabile in modo deterministico. Per questo il baseline integrato aggiunge un requisito di industrializzazione: **ogni modello candidabile a deployment deve nascere con bundle frozen nativo.**

26.2 Artifact obbligatori

Il bundle frozen deve serializzare almeno:

- modello long e modello short, se distinti;
- stato fitted del preprocessor;
- schema delle feature con ordine persistito;
- soglie di inference;
- configurazione delle guardie;
- filtri attivi;
- regole di uscita attive;
- versione dei modelli di volatilità e covarianza;
- policy di fallback.

26.3 Perché è essenziale

Senza questi artifact, il replay OOS e la ricostruzione del frontier congelato diventano intrinsecamente fragili. Un motore genetico che non sia in grado di preservare la propria inferenza frozen non può essere considerato validabile in senso forte.

26.4 Relazione con la no-refresh doctrine

Il bundle frozen e il no-refresh appartengono alla stessa famiglia di requisiti: entrambi impediscono che il sistema cambi retrospettivamente il proprio significato operativo. Il primo lo fa a livello di modello, il secondo a livello di segnale emesso.

Testabilità e tracciabilità

27.1 Classi minime di test

Il piano di test eseguibile deve coprire almeno:

- semantica dei dati e timestamp;
- orientamento dei prezzi LONG/SHORT;
- semantica conservativa di touch/target/invalidation;
- assenza di touch inferiti su gap;
- costruzione del tensore di feature;
- monotonicità di ampiezza e invalidazione rispetto al rischio;
- stimatori di volatilità e PSD della covarianza;
- regole di invalidazione di regime;
- cromosoma e dominance del GA;
- immutabilità del core hash;
- recovery post-restart;
- eseguibilità solo a touch reale;
- precedenza eventi a timestamp identico;
- enforcement degli split temporali ordinati;
- reporting robusto per regime.

27.2 Tracciabilità requisito -> test

Il punto non è produrre una lista di test isolati, ma una vera matrice di tracciabilità. Ogni requisito normativo deve avere almeno un test associato e ogni test deve ricondursi a un requisito. Questo rende il documento auditabile e trasforma la monografia in una base di QA, non solo di design.

Architettura finale del motore come sintesi concettuale

Il motore finale può essere compreso come un sistema a quattro blocchi accoppiati:

1. **blocco di rischio:** modelli univariati e multivariati di varianza/covarianza condizionale;
2. **blocco di geometria del contratto:** quantili condizionali per entry, target, durata e invalidazione;
3. **blocco di sopravvivenza decisionale:** survival pre-entry e post-entry con rischi concorrenti;
4. **blocco evolutivo e validativo:** GA multi-obiettivo con dominance vincolata e filtri anti-overfitting.

Se uno qualunque di questi blocchi viene sostituito da una scorciatoia grezza, l'intero motore perde coerenza:

- senza il blocco di rischio, la zona non è eteroschedastica;
- senza la geometria del contratto, il segnale non è un oggetto persistente;
- senza la survival analysis, il touch viene ridotto a classificazione sbagliata;
- senza il blocco evolutivo-validativo, il sistema diventa un ottimizzatore di backtest.

Conclusione

Il modello qui formalizzato non è un semplice “motore di segnali”. È un **motore genetico strutturale per contratti runtime persistenti**, in cui la decisione di pubblicare una zona futura è già una decisione forte, ma distinta dalla decisione di eseguire al touch. La distinzione fra queste due fasi è il cuore metodologico dell’intero impianto.

La monografia consolida in forma unica e non conversazionale gli elementi che nei sorgenti erano dispersi in specifiche, approfondimenti e note di ricerca. Il risultato è un baseline che può fungere contemporaneamente da:

- documento di riferimento teorico;
- specifica per implementazione;
- contratto di test;
- base per audit quantitativo;
- fondamento per industrializzazione e run real-time controllato.

Boundary implementativi: da specifica a motore eseguibile

30.1 Separazione dei layer

Per evitare che la teoria venga corrotta in implementazione, il motore deve essere organizzato in layer separati:

- **quant**: stima dei modelli di volatilità/covarianza, quantili, hazard e geometria della zona;
- **runtime**: gestione del lifecycle del segnale, guardie, eventi, ordini e replay;
- **validation**: pipeline walk-forward, metriche, DSR, PBO e gate di deploy;
- **reports**: produzione di report fold-by-fold, per regime e per indice;
- **registry/artifacts**: serializzazione del bundle frozen e versionamento delle specifiche.

Questa separazione impedisce che logica di ricerca, logica runtime e logica di validazione collassino in un unico modulo opaco.

30.2 Confine fra research semantics e runtime semantics

Una delle principali fonti di errore nei sistemi quantitativi è la divergenza tra ciò che il backtest assume e ciò che il runtime fa davvero. Nel motore qui formalizzato deve valere:

$$\text{research semantics} = \text{runtime semantics}.$$

In termini concreti, significa che:

- il validatore OOS deve usare il replay della stessa state machine che userà il runtime;
- le metriche devono essere calcolate sugli stessi eventi e sulle stesse precedenze;
- il motore di ricerca non può inventare semantiche di touch, fill o invalidazione diverse da quelle operative.

30.3 Frozen bundle e versionamento

Ogni artefatto di inferenza deve essere versionato con almeno tre livelli di identificazione:

1. versione del documento/specifica;
2. versione del bundle frozen;
3. versione del segnale emesso.

Questo consente di ricostruire non solo il modello, ma anche il quadro normativo e semantico sotto cui il modello operava.

30.4 Ledger delle decisioni

Per ogni fold e per ogni segnale emesso, il sistema deve poter produrre un ledger minimale con:

- timestamp evento;
- stato precedente e successivo;
- cause della transizione;
- snapshot delle guardie;
- riferimento al bundle frozen;
- riferimento al feature snapshot;
- riferimento all'ordine, se esistente.

Questo ledger è essenziale sia per l'audit quantitativo sia per l'analisi di failure mode.

Protocollo operativo dalla ricerca al run controllato

31.1 Sequenza corretta

La transizione dalla ricerca alla prova storica reale dovrebbe seguire questa sequenza:

1. congelamento della semantica e dei requisiti;
2. congelamento del test plan eseguibile;
3. definizione dei contratti API e dei boundary fra moduli;
4. formalizzazione della state machine runtime;
5. definizione del protocollo walk-forward con DSR/PBO;
6. consolidamento del package con namespace stabili;
7. test end-to-end sintetico;
8. primo run storico controllato con reporting per fold e per regime;
9. solo dopo, eventuale candidate-live.

31.2 Errori da evitare

Gli errori più pericolosi in questa fase sono:

- fare tuning cieco su metriche aggregate senza audit del lifecycle;
- cambiare la semantica del segnale mentre si cambiano i modelli;
- usare OOS non riproducibili perché privi di bundle frozen;
- giudicare il motore solo su PnL e non sulla calibrazione delle probabilità e sulla stabilità per regime.

31.3 Candidate-live gate

Una specifica può essere candidata al live controllato solo se, simultaneamente:

- supera i gate minimi di Π^{exe} e Π^{E2E} ;
- mantiene PBO sotto soglia;

- ha DSR accettabile;
- non mostra breakdown semantici nel replay;
- possiede bundle frozen ricostruibile;
- produce report per regime privi di collassi sistematici.

Pseudocodice di alto livello

32.1 Training

```
INPUT: market_data, target_catalog, config_space, outer_splits

for split in outer_splits:
    train, valid, test = temporal_partition(split, purge=True, embargo=True)

    for chromosome in population:
        fit_univariate_vol_models(train, chromosome.vol_family)
        standardize_residuals(train)
        fit_cross_index_covariance(train, chromosome.cov_family)

        build_feature_tensors(train, valid, test, chromosome.feature_subset)
        fit_quantile_models(train, chromosome.quantile_params)
        fit_pre_entry_hazard(train, chromosome.guard_policy)
        fit_post_entry_hazard(train, chromosome.exit_policy)

        replay_valid = simulate_signal_lifecycle(valid, chromosome)
        score_valid = evaluate_multiobjective_metrics(replay_valid)

    evolve_population_via_constrained_nsga2()

select_pareto_candidates()
run_outer_test_replay_with_frozen_bundle()
compute_DSR_PBO_and_regime_reports()
OUTPUT: frozen_bundle_candidates, diagnostics, reports
```

32.2 Emission inference

```
INPUT: latest_state, frozen_bundle, target_catalog
```

```

X_t = build_feature_tensor(latest_state)
H_t = estimate_conditional_covariance(X_t, frozen_bundle)
candidates = enumerate_structural_targets(target_catalog)

for candidate in candidates:
    center = compute_zone_center(X_t, candidate, frozen_bundle)
    half_width = compute_half_width(X_t, H_t, frozen_bundle)
    ttl = compute_valid_until(X_t, candidate, frozen_bundle)
    invalidation = compute_invalidation(center, half_width, H_t, frozen_bundle)

    feasibility = evaluate_publish_filters(X_t, H_t, candidate, frozen_bundle)

    if feasibility.pass_all:
        signal = persist_frozen_signal(...)
        publish(signal)

```

32.3 Runtime

INPUT: active_signals, market_event

```

for signal in active_signals:
    if signal.state == PENDING:
        apply_event_precedence(signal, market_event)
        if raw_touch(signal, market_event):
            outcome = evaluate_runtime_guardrails(signal, market_event)
            transition(signal, outcome)
    elif signal.state == EXECUTABLE:
        submit_order_if_allowed(signal)
    elif signal.state == ORDER_WORKING:
        consume_order_events(signal)

```

Scheletro della suite di test

33.1 Suite dati e semantica

- test serializzazione UTC;
- test orientamento LONG/SHORT;
- test tick alignment;
- test touch conservativo;
- test assenza di touch latenti su gap.

33.2 Suite quantitativa

- test presenza dei canali minimi nel tensore;
- test monotonicità di ampiezza e invalidazione;
- test PSD di H_t ;
- test blocco deployment con ARCH residui;
- test coerenza fra σ_{issued} e guardie di regime.

33.3 Suite runtime

- test precedenza expiry > invalidation > target-before-entry > touch;
- test touch con sole soft guard fallite;
- test hard reject;
- test massimo un ordine per segnale;
- test recovery post-restart senza duplicazioni;
- test cancel on expiry in ORDER_WORKING.

33.4 Suite validation

- test split temporali ordinati;
- test purge ed embargo;
- test DSR e PBO su fixture controllate;

- test reporting per regime;
- test gate globali PASS / CONDITIONAL / FAIL.

Decisioni che questa monografia congela in modo definitivo

1. Il segnale è un contratto runtime persistente e non un output effimero.
2. I campi core non si aggiornano dopo la pubblicazione.
3. L'eseguibilità è valutata solo al touch reale.
4. Il target strutturale è un evento assorbente definito prima dell'esecuzione.
5. L'eteroschedasticità è parte sia della generazione sia della persistenza del segnale.
6. La covarianza condizionale cross-index è modellata esplicitamente.
7. Il touch è trattato come survival problem con competing risks.
8. La validazione è strettamente time-ordered e anti-overfitting.
9. Il deployment richiede un bundle frozen completo e ricostruibile.
10. La semantica usata in ricerca deve coincidere con la semantica runtime.

Glossario dei campi e dei simboli

A.1 Campi principali del segnale

Campo / simbolo	Significato operativo
<code>signal_id</code>	Identificatore univoco immutabile del segnale.
<code>instrument_id</code>	Codice canonico dell'indice bersaglio.
s	Lato operativo, con +1 per LONG e -1 per SHORT.
<code>issued_at</code>	Istante di emissione del contratto.
<code>valid_from</code>	Estremo inferiore della validità temporale.
<code>valid_until</code>	Estremo superiore esclusivo della validità temporale.
$[e_L, e_H]$	Entry zone tick-aligned.
$[g_L, g_H]$	Target zone tick-aligned.
I	Barriera hard di invalidazione pre-entry.
X_t	Tensore multidimensionale di feature.
H_t	Matrice di covarianza condizionale.
$\sigma_{i,t}$	Volatilità condizionale univariata dell'indice i .
$\sigma_t^{sys}(u)$	Stress sistemico proiettato lungo la direzione u .
h_c	Core hash dei campi immutabili.

A.2 Stati principali

Stato	Semantica
PENDING	Segnale persistito e congelato, in attesa di touch valido.
EXECUTABLE	Touch osservato e guardie passate; il segnale può generare ordine.
ORDER_WORKING	Ordine accettato dalla venue e non ancora completamente eseguito.
EXECUTED	Ordine eseguito.

Stato	Semantica
EXPIRED	Validità temporale terminata prima dell'esecuzione.
INVALIDATED	Invalidazione hard per prezzo o regime.
MISSED_TARGET	Target raggiunto prima della zona di ingresso.
REJECTED_TOUCH	Touch osservato ma rifiutato per guardie hard o pretrade gate.
ORDER_REJECTED	Ordine rifiutato dalla venue.
CANCELLED_UNFILLED	Ordine cancellato senza fill.

Catalogo sintetico dei requisiti atomici

B.1 Requisiti dati e semantica

- Tutti i timestamp core devono essere UTC con precisione almeno al microsecondo.
- Entry zone e target zone devono essere intervalli chiusi, tick-aligned e con vincolo $\text{low} \leq \text{high}$.
- Ogni segnale con orientamento incoerente LONG/SHORT deve essere rifiutato.
- Touch, target e invalidazione devono usare quote conservative coerenti con il lato.
- In backtest bar-based non è ammesso inferire touch latenti senza ricostruzione intrabar certificata.

B.2 Requisiti modello multidimensionale

- All'emissione deve esistere un tensore X_t con i canali minimi obbligatori.
- Il vettore Y_t deve includere distanza, durata e misure di escursione.
- Il centro della zona deve essere una quantile prediction.
- L'ampiezza deve essere funzione monotona non decrescente di volatilità, covarianza, spread e illiquidità.
- `valid_until` deve derivare da una distribuzione di durata al touch.
- Il segnale deve essere pubblicato solo se tutte le soglie di fattibilità sono soddisfatte.

B.3 Requisiti volatilità/covarianza

- Ogni canale residuo deve avere varianza condizionale tempo-variabile.
- La matrice H_t deve essere PSD e con correlazioni tempo-variabili.
- Deve esistere una regola hard di volatility regime break.
- Una specifica che lasci ARCH residui oltre soglia o perda PSD non è deployable.

B.4 Requisiti motore genetico

- Il cromosoma deve codificare feature subset, modelli di volatilità e covarianza, parametri di zona, durata, soglie di fattibilità e guardie.
- La fitness deve essere multi-obiettivo e includere hit rate esecutivo, expected return, CVaR, stabilità per regime e penalità per zone mai toccate.
- L'intera valutazione del GA deve essere OOS time-ordered.

B.5 Requisiti persistenza ed esecuzione

- Il segnale deve essere persistito con core hash.
- I campi core sono immutabili finché il segnale è attivo.
- Il retraining influenza solo segnali futuri.
- L'eseguibilità viene valutata solo al raw touch reale, entro validità e senza invalidazioni precedenti.
- Le guardie runtime devono distinguere fra soft e hard.
- È ammesso al massimo un ordine per segnale.

Matrice sintetica delle metriche

Famiglia	Metrica	Funzione
Lifecycle	Raw touch rate	Misura la capacità della zona di essere effettivamente visitata.
Lifecycle	Executable rate	Misura quanto spesso il raw touch diventa eseguibile.
Lifecycle	Invalidation rate	Misura la fragilità del contratto rispetto a prezzo/regime.
Performance	Expected net return	Valuta l'utilità economica condizionata all'esecuzione.
Rischio	CVaR	Misura il tail risk della distribuzione dei ritorni.
Calibrazione	Brier score	Misura la calibrazione di probabilità di eseguibilità/target.
Robustezza	Variance by regime	Penalizza instabilità fra regimi e indici.
Anti-overfitting	DSR	Deflaziona Sharpe per selection bias e multiple prove.
Anti-overfitting	PBO	Stima la probabilità che il miglior backtest sia overfit.
Deployment	Pass/Fail gate	Stabilisce se una specifica è candidabile al live controllato.

Contratto minimo di industrializzazione

Per portare il modello da baseline metodologica a motore reale, il package di produzione deve includere:

- serializzazione nativa di modelli, preprocessori, feature schema e soglie;
- event log replayable;
- recovery post-restart con idempotenza ordini;
- audit trail che colleghi segnale, feature snapshot, modello e decisione;
- separazione dei namespace tra layer quant, runtime, validation e reporting;
- test end-to-end su dataset sintetico prima dei run storici reali;
- ledger di decisioni per fold e per regime.

Appendice dimostrativa hard-locked: lemmi, teoremi, proposizioni e sketch di dimostrazione

E.1 Proposizione 1 — Il tempo di raw touch è uno stopping time

Sia P_t^{touch} adattato alla filtrazione evento-driven $(\mathcal{F}_t)$ e siano $e_L, e_H, t_{min}, t_{max}$ campi core fissati al tempo di emissione t_0 . Definiamo:

$$\tau_{touch} = \inf\{t \geq t_0 : P_t^{touch} \in [e_L, e_H], t \in [t_{min}, t_{max}), state_{t-} = PENDING\}.$$

Allora τ_{touch} è uno stopping time rispetto a $(\mathcal{F}_t)$.

Sketch di dimostrazione. L'evento $\{\tau_{touch} \leq t\}$ dipende soltanto dall'osservazione del processo adattato fino al tempo t , dalla conoscenza di campi core già fissati a t_0 e dallo stato runtime, anch'esso funzione del ledger fino a t . Quindi $\{\tau_{touch} \leq t\} \in \mathcal{F}_t$.

E.2 Proposizione 2 — Minimo di tempi terminali concorrenti

Siano $\tau_{touch}, \tau_{inv}, \tau_{exp}, \tau_{miss}$ stopping times del lifecycle pre-entry. Allora

$$\tau_{term}^{pre} = \tau_{touch} \wedge \tau_{inv} \wedge \tau_{exp} \wedge \tau_{miss}$$

è anch'esso uno stopping time.

Sketch. Il minimo finito di stopping times è uno stopping time. Questa proprietà giustifica la trattazione del problema come first-event problem con rischi concorrenti.

E.3 Proposizione 3 — Non negatività dello stress sistemico proiettato

Se $H_t \succeq 0$ e u è un vettore di esposizione deterministico o $\mathcal{F}_t$ -misurabile, allora:

$$\sigma_t^{sys}(u) = \sqrt{u^\top H_t u}$$

è ben definita e non negativa.

Sketch. Per PSD di H_t , ogni forma quadratica $u^\top H_t u$ è non negativa; la radice quadrata è dunque ben definita nei reali non negativi.

E.4 Proposizione 4 — Monotonia della semilarghezza della zona

Se $k_1, k_2, k_3, k_4 \geq 0$, allora la semilarghezza

$$w_t = \text{clip}(w_{min}, w_{max}, k_1 \sigma_{i,t} + k_2 \sigma_t^{sys}(u) + k_3 \text{spread}_t + k_4 \text{depth}_t^{-1})$$

è monotona non decrescente in ciascuno degli argomenti di rischio/frizione.

Sketch. La combinazione lineare con coefficienti non negativi è monotona in ciascun argomento. L'operatore `clip` preserva la monotonia, limitandosi a saturare il valore a soglie inferiori e superiori.

E.5 Proposizione 5 — Invarianza del core sotto retraining forward-only

Sia $Core(S_t)$ l'insieme dei campi core di un segnale attivo emesso al tempo t_0 . Se il sistema impone forward-only retraining e vieta modifiche dei campi core in stati attivi, allora per ogni $t \geq t_0$ finché il segnale non è terminale:

$$Core(S_t) = Core(S_{t_0}).$$

Sketch. Per ipotesi il retraining può influire solo su segnali futuri e il runtime può modificare solo campi di audit/stato. Nessuna transizione ammessa dalla macchina modifica i campi core; quindi l'insieme resta invariato.

E.6 Proposizione 6 — Probabilità end-to-end come decomposizione di survival

Sia $S(k)$ la survival pre-entry e sia $h^{exe}(k)$ la hazard di eseguibilità. Sia inoltre $\Pi^{tar|exe}(H - k)$ la probabilità di target post-entry entro orizzonte residuo. Allora

$$\Pi^{E2E}(K, H) = \sum_{k=1}^K S(k-1) h^{exe}(k) \Pi^{tar|exe}(H-k).$$

Sketch. La formula segue dalla decomposizione della probabilità totale sugli istanti di prima eseguibilità. Ogni termine somma la probabilità di sopravvivere fino a $k-1$, diventare eseguibile in k e poi raggiungere il target nel post-entry residuo.

E.7 Proposizione 7 — Well-definedness del DCC baseline

Se nella ricorrenza DCC valgono $a \geq 0$, $b \geq 0$, $a+b < 1$ e la matrice di lungo periodo $\bar{Q}$ è definita positiva, allora la successione Q_t resta nel cono delle matrici simmetriche positive definite sotto condizioni standard sulla stima dei residui standardizzati; conseguentemente la normalizzazione produce R_t con diagonale unitaria e PSD.

Sketch. La ricorrenza è una combinazione convessa di una matrice PD e di contributi semidefiniti positivi; la positività stretta della diagonale consente la normalizzazione. In pratica la stima numerica richiede inoltre shrinkage e soglie ϵ per garantire stabilità computazionale.

E.8 Proposizione 8 — BEKK e positiva definitezza strutturale

Nella parametrizzazione BEKK

$$H_t = CC^\top + A^\top \varepsilon_{t-1} \varepsilon_{t-1}^\top A + B^\top H_{t-1} B,$$

se C ha rango pieno e $H_{t-1} \succ 0$, allora $H_t \succ 0$.

Sketch. Il termine $CC^\top$ è PD, gli altri due termini sono PSD. La somma di una matrice PD e di matrici PSD è PD.

E.9 Lemma 1 — Compattezza dello spazio dei cromosomi feasible

Sia $\mathcal{X}$ lo spazio dei cromosomi, ottenuto come prodotto di un insieme finito di geni discreti e di intervalli chiusi e limitati per i geni continui. Sia inoltre $\mathcal{X}_F \subseteq \mathcal{X}$ l'insieme dei cromosomi feasible dopo applicazione dei vincoli chiusi del baseline (domini, monotonia, soglie, complessità massima, deployability minima). Allora $\mathcal{X}_F$ è compatto.

Sketch. Il prodotto di un insieme finito e di compatti è compatto. I vincoli del baseline definiscono sottoinsiemi chiusi del prodotto. Un chiuso di un compatto è compatto; quindi $\mathcal{X}_F$ è compatto.

E.10 Teorema 1 — Esistenza del fronte Pareto feasible e di almeno un bundle congelabile

Sia $F : \mathcal{X}_F \rightarrow \mathbb{R}^m$ la mappa multi-obiettivo outer-valid/test, con componenti continue o almeno semicontinue inferiormente sul compatto $\mathcal{X}_F$. Allora l'insieme dei punti Pareto-feasible non è

vuoto. Se inoltre la governance quantitativa finale usa un ordinamento lessicografico totale su $(GO/CONDITIONAL/NO-GO, DSR, PBO, Robustness, Complexity)$, esiste almeno un bundle frozen selezionabile in modo deterministico.

Sketch. L'immagine $F(\mathcal{X}_F)$ di un compatto tramite mappa continua è compatta; un compatto non vuoto in $\mathbb{R}^m$ possiede sempre almeno un elemento non dominato. L'ordinamento lessicografico totale sul sottoinsieme archiviato seleziona poi un elemento univoco o un insieme finito di ex-aequo ulteriormente risolti da tie-breaker congelati; in ogni caso esiste almeno un bundle congelabile.

E.11 Lemma 2 — Dipendenza piecewise-Lipschitz della geometria della zona

Supponiamo che i quantili condizionali usati per c_t e t_{max} , così come gli stimatori di $\sigma_{i,t}$, $\sigma_t^{sys}(u)$, $spread_t$ e $depth_t^{-1}$, siano localmente Lipschitz fuori da un insieme di discontinuità di misura nulla e che l'operatore `clip` abbia soglie finite. Allora le mappe

$$\xi_t \mapsto c_t, \quad \xi_t \mapsto w_t, \quad \xi_t \mapsto I_t, \quad \xi_t \mapsto t_{max}$$

sono piecewise-Lipschitz in $\xi_t = (X_t, \sigma_{i,t}, H_t, spread_t, depth_t)$.

Sketch. Somme, prodotti per costanti e composizioni di mappe Lipschitz restano Lipschitz. L'operatore `clip` è 1-Lipschitz e introduce solo regioni di saturazione. Ne segue una dipendenza piecewise-Lipschitz della geometria del contratto rispetto allo stato osservato.

E.12 Teorema 2 — Non-degenerazione minima del landscape di fitness

Assumiamo:

1. $\mathcal{X}_F$ compatto e non vuoto;
2. esistenza di almeno due cromosomi feasible $\chi_a, \chi_b \in \mathcal{X}_F$ che differiscono in almeno un gene con effetto reale sul layer quant, sul layer runtime o sulla penalità di complessità;
3. esistenza di un insieme di stati di mercato di misura positiva su cui tale differenza di gene modifica almeno uno tra c_t , w_t , I_t , t_{max} , una probabilità di hazard oppure il termine di complessità;
4. obiettivi di fitness che includono almeno una componente tra calibrazione, rischio, robustezza o complessità non costante per costruzione.

Allora la mappa di fitness multi-obiettivo $F(\chi)$ non è costante su $\mathcal{X}_F$. In particolare il landscape feasible è **minimamente non degenero**: esiste almeno una coppia (χ_a, χ_b) tale che $F(\chi_a) \neq F(\chi_b)$.

Sketch. Procediamo per assurdo. Se F fosse costante su $\mathcal{X}_F$, ogni coppia di cromosomi feasible avrebbe identico vettore di rischio, calibrazione, robustezza e complessità. Ma per l'assunzione 2–3 esiste almeno un gene che modifica su un insieme di stati di misura positiva la geometria del contratto, una hazard o il termine di complessità. Tale modifica altera almeno una componente di

F : direttamente la complessità, oppure indirettamente le metriche lifecycle/rischio/calibrazione calcolate sul replay. Si ottiene una contraddizione. Dunque il landscape non è piatto sullo spazio feasible.

E.13 Corollario 1 — Significato operativo del plateau di hypervolume

Sotto le ipotesi del Teorema 2, il verificarsi del criterio

$$\max_{j=g-G_{stall}+1, \dots, g} |HV_j - HV_{j-1}| \leq \epsilon_{HV}$$

non può essere interpretato come conseguenza banale di un landscape completamente piatto, ma come indicazione di stagnazione algoritmica alla risoluzione ϵ_{HV} del baseline.

Sketch. Se il landscape feasible non è costante, l'assenza di miglioramento dell'hypervolume per un numero sufficiente di generazioni segnala che la ricerca non sta più trovando miglioramenti apprezzabili alla scala ϵ_{HV} , non che non esistano differenze di fitness tra architetture.

Piano di test eseguibile completo e criteri di accettazione

F.1 Obiettivo del piano di test

Il piano di test non è un allegato accessorio. È il secondo contratto normativo del motore dopo la semantica del segnale. Il suo scopo è impedire che implementazioni diverse reinterpretino in modo opportunistico touch, validità, persistenza, hazard, rischio sistemico e regole di validazione. Ogni requisito del modello deve quindi avere almeno uno scenario eseguibile che lo renda falsificabile.

Il piano integrato usa due forme complementari:

- scenari leggibili in formato **Given / When / Then** per audit umano;
- traduzione uno-a-uno in **suite pytest** eseguibili nel package del motore.

F.2 Struttura minima della suite

La suite integrata deve essere organizzata in namespace stabili, con naming coerente con i blocchi del motore:

- `tests/test_signal_semantics.py`
- `tests/test_execution_gate.py`
- `tests/test_volatility_layer.py`
- `tests/test_competing_risks.py`
- `tests/test_post_entry.py`
- `tests/test_ga_search.py`
- `tests/test_temporal_protocol.py`
- `tests/test_runtime_edgecases.py`
- `tests/test_validation_gates.py`
- `tests/test_bundle_reproducibility.py`

Ogni file di test deve dipendere da fixture minimali e deterministiche, non da servizi esterni o da dati di mercato non congelati.

F.3 Fixture obbligatorie

Il baseline integrato congela le seguenti fixture minime:

- `fixture_signal_long()` e `fixture_signal_short()` con campi core completi e core hash coerente;
- `fixture_market_path_touch_soft_fail()` con touch osservato e fallimento di sole guardie soft;
- `fixture_market_path_touch_hard_fail()` con touch osservato e hard reject;
- `fixture_market_path_target_before_entry()` con target raggiunto prima della entry zone;
- `fixture_market_path_vol_regime_break()` con shock di volatilità/covarianza sufficiente a invalidare il segnale;
- `fixture_order_events_fill_before_cancel()` per il caso di precedenza `fill > cancel`;
- `fixture_walk_forward_splits()` con fold time-ordered, purge ed embargo già applicati;
- `fixture_frozen_bundle_complete()` e `fixture_frozen_bundle_incomplete()` per i test di ricostruibilità.

Le fixture sintetiche devono essere piccole ma semanticamente ricche: la loro funzione non è imitare tutto il mercato, ma rendere osservabile il comportamento corretto del motore.

F.4 Catalogo scenari Given / When / Then

F.4.1 Suite dati e semantica

SCN-DAT-001 — serializzazione UTC e ordinamento temporale - **Given** un segnale con `issued_at`, `valid_from`, `valid_until` e un flusso di eventi tutti in UTC; - **When** il dispatcher ordina gli eventi e valuta la validità temporale; - **Then** l'ordinamento è deterministico e nessun confronto temporale dipende da timezone implicite.

SCN-DAT-002 — tick alignment delle zone - **Given** una entry zone e una target zone con prezzi non allineati al tick; - **When** il segnale viene validato prima della persistenza; - **Then** il sistema rifiuta il segnale o lo normalizza secondo la policy congelata, senza lasciare stati ambigui.

SCN-DAT-003 — orientamento LONG coerente - **Given** un segnale LONG con `invalidation_price < entry_zone_low <= entry_zone_high < target_zone_low <= target_zone_high`; - **When** il validatore semantico controlla il contratto; - **Then** il segnale è semanticamente ammissibile.

SCN-DAT-004 — orientamento incoerente - **Given** un segnale con target e entry invertiti rispetto al lato dichiarato; - **When** il validatore del core esegue i controlli di orientamento; - **Then** il segnale viene rifiutato prima di entrare nella pipeline quantitativa.

SCN-DAT-005 — touch conservativo su quote corrette - **Given** un segnale LONG e quote bid/ask coerenti con la microstruttura; - **When** il prezzo ask entra nella entry zone ma il bid no; - **Then** il sistema dichiara touch secondo la sorgente `BEST_ASK` e non usa riferimenti più favorevoli ma non negoziabili.

SCN-DAT-006 — divieto di touch latente su gap - **Given** una barra che apre sotto la

zona e chiude sopra senza ricostruzione intrabar; - **When** il replay valuta il touch; - **Then** nessun touch viene inferito per semplice attraversamento implicito del range.

F.4.2 Suite quantitativa e layer di rischio

SCN-QNT-001 — **presenza dei canali minimi nel tensore** - **Given** un dataset di training e un subset di feature candidato; - **When** il costruttore del tensore genera X_t ; - **Then** returns, realized volatility, vol-of-vol, spread/liquidità, relazioni cross-index, struttura e regime sono presenti o il build fallisce.

SCN-QNT-002 — **monotonicità dell'ampiezza della zona** - **Given** due stati identici salvo un aumento di volatilità locale o stress sistemico; - **When** il kernel `compute_half_width` produce w_t ; - **Then** l'ampiezza della zona non diminuisce.

SCN-QNT-003 — **validità temporale da quantile di durata** - **Given** un modello di durata al touch e un segnale candidato; - **When** il sistema calcola `valid_until`; - **Then** `valid_until` deriva da un quantile della distribuzione di τ_e e non da un TTL fisso in barre.

SCN-QNT-004 — **invalidazione crescente col rischio** - **Given** due stati con stessa zona ma rischio sistemico differente; - **When** il kernel `compute_invalidation` produce I_t ; - **Then** la distanza dell'invalidation price aumenta con il rischio secondo la forma congelata.

SCN-QNT-005 — **PSD enforcement della covarianza** - **Given** una stima di H_t quasi singolare o numericamente instabile; - **When** il layer di covarianza applica i controlli PSD; - **Then** il sistema attiva la policy congelata di fallback o blocca il deploy della specifica.

SCN-QNT-006 — **volatility regime break hard** - **Given** uno shock che porta $\sigma_{p,\tau}/\sigma_{p,issued}$ o $\lambda_{max}(H_\tau)/\lambda_{max}(H_{issued})$ oltre soglia; - **When** il segnale è ancora PENDING; - **Then** il lifecycle passa a INVALIDATED con reason code coerente.

F.4.3 Suite runtime e macchina a stati

SCN-RTM-001 — **touch con sole guardie soft fallite** - **Given** un raw touch reale entro validità e un set di guardie dove falliscono solo condizioni soft; - **When** il dispatcher valuta il touch; - **Then** `touch_count` aumenta, viene scritto uno snapshot runtime e il segnale resta PENDING.

SCN-RTM-002 — **touch con hard reject** - **Given** un raw touch reale e una hard guard fallita; - **When** il dispatcher valuta le guardie; - **Then** il segnale transita a REJECTED_TOUCH o INVALIDATED secondo la natura della guardia.

SCN-RTM-003 — **touch eseguibile** - **Given** un raw touch reale e tutte le guardie superate; - **When** il dispatcher processa l'evento; - **Then** il segnale passa a EXECUTABLE e non direttamente a ORDER_WORKING.

SCN-RTM-004 — **execution window missed** - **Given** un segnale già EXECUTABLE e nessun ordine inviato entro la finestra esecutiva massima Δ_{exe} ; - **When** arriva il clock event di scadenza della execution window; - **Then** il segnale passa a EXECUTION_WINDOW_MISSED.

SCN-RTM-005 — **cancel on expiry in ORDER_WORKING** - **Given** un ordine già ORDER_WORKING al momento del raggiungimento di `valid_until`; - **When** si attiva la cancel

rule congelata; - **Then** il sistema alza `cancel_requested` e chiude in `CANCELLED_UNFILLED` solo quando l'esito ordine è coerente.

SCN-RTM-006 — precedenza fill > cancel - **Given** un fill e un cancel con timestamp identico; - **When** il dispatcher applica la precedenza eventi; - **Then** il fill prevale e il lifecycle termina in `EXECUTED`.

SCN-RTM-007 — restart idempotente - **Given** un riavvio del runtime con segnali attivi e ledger persistito; - **When** il sistema ricostruisce lo stato; - **Then** nessun ordine viene duplicato e le transizioni già consumate non vengono rieseguite.

SCN-RTM-008 — unsupported event - **Given** un evento fuori dominio rispetto al dispatcher congelato; - **When** il runtime lo riceve; - **Then** l'evento viene rifiutato esplicitamente e registrato nel ledger con `UNSUPPORTED_EVENT`.

F.4.4 Suite survival e rischi concorrenti

SCN-SRV-001 — raw touch come hazard discreto - **Given** una sequenza di covariate grouped/event-driven e un modello cloglog di raw touch; - **When** viene calcolata $h^{touch}(k | x_k)$; - **Then** il valore è in $(0, 1)$ e produce una probabilità cumulativa coerente entro expiry.

SCN-SRV-002 — fattorizzazione pass / soft / hard - **Given** un raw touch al tempo k e covariate di guardia g_k ; - **When** il motore calcola $p^{pass}(k)$, $p^{hard}(k)$ e $p^{soft}(k)$; - **Then** le tre componenti sono non negative e sommano a uno.

SCN-SRV-003 — target before entry come rischio concorrente - **Given** una traiettoria che raggiunge la target zone prima della entry zone; - **When** il replay aggiorna la survival del pending signal; - **Then** l'evento viene contato come `MISSED_TARGET` e non come successo end-to-end.

SCN-SRV-004 — probabilità end-to-end - **Given** un bundle di hazard pre-entry e post-entry stimato sul fold; - **When** il validatore calcola $\Pi^{E2E}(K, H)$; - **Then** la probabilità end-to-end è coerente con la decomposizione emissione -> eseguibilità -> target.

F.4.5 Suite validazione e anti-overfitting

SCN-VAL-001 — split temporali ordinati - **Given** una collezione di fold walk-forward; - **When** il validatore costruisce train, valid e test; - **Then** nessun record del test entra causalmente nel train o nella scelta delle soglie.

SCN-VAL-002 — purge ed embargo - **Given** feature o label con finestre temporali sovrapposte; - **When** il validatore costruisce i fold; - **Then** train e test vengono separati con purge ed embargo coerenti con l'orizzonte massimo delle dipendenze.

SCN-VAL-003 — fit dei cutpoint di regime solo sul train - **Given** una metrica di regime R_t e un fold corrente; - **When** il sistema stima i cutpoint di regime; - **Then** i quantili sono calcolati solo sul train del fold e restano congelati su valid e test.

SCN-VAL-004 — DSR con numero effettivo di prove - **Given** una popolazione GA e un insieme di risultati OOS dipendenti; - **When** il motore calcola il DSR; - **Then** il parametro `n_effective_trials` non coincide banalmente con il numero totale di cromosomi valutati.

SCN-VAL-005 — PBO tramite CSCV/CPCV - Given una collezione di performance ordinate per combinazioni di fold; - **When** il validatore esegue CSCV o CPCV; - **Then** produce una stima di PBO coerente con la perdita di ranking tra train e test.

SCN-VAL-006 — gate di fold - Given metriche lifecycle, rischio, calibrazione, DSR, PBO e robustezza di un fold; - **When** il gate del fold viene applicato; - **Then** il fold viene classificato in PASS, CONDITIONAL o FAIL senza ambiguità.

SCN-VAL-007 — decisione globale - Given tutti i fold outer di una specifica; - **When** il validatore aggrega i risultati; - **Then** la decisione finale è GO, CONDITIONAL o NO-GO secondo la regola congelata.

F.4.6 Suite frozen bundle e riproducibilità

SCN-BND-001 — manifest completo - Given un bundle candidato al deploy; - **When** il checker del manifest verifica gli artefatti obbligatori; - **Then** modello, preprocessor, schema feature, layer rischio, guardie, metriche e ambiente devono essere tutti presenti e versionati.

SCN-BND-002 — replay deterministico entro tolleranza - Given stesso bundle frozen, stessi input e stesso event stream; - **When** il motore riesegue l'inference e il replay; - **Then** i segnali, gli stati e le metriche coincidono entro tolleranza definita.

SCN-BND-003 — hash chain bundle -> features -> core - Given un segnale emesso; - **When** il sistema costruisce la hash chain di audit; - **Then** ogni decisione runtime resta riconducibile a bundle, features e campi core della versione corretta.

F.5 Traduzione minima in pytest

Ogni scenario deve avere una controparte eseguibile con struttura semplice e leggibile. La forma minima attesa è la seguente:

```
def test_touch_with_only_soft_failures_keeps_pending(fixture_signal_long,
↳ fixture_market_path_touch_soft_fail):
    signal = fixture_signal_long
    events = fixture_market_path_touch_soft_fail

    final_signal, ledger = replay_signal(signal, events)

    assert final_signal.state == "PENDING"
    assert final_signal.touch_count == 1
    assert ledger[-1].reason_code == "SOFT_GUARD_FAIL"
```

Per i test di validazione, la forma minima è:

```
def test_fold_gate_returns_conditional_when_dsr_is_marginal(sample_fold_metrics):
    decision = classify_fold(sample_fold_metrics)
    assert decision in {"PASS", "CONDITIONAL", "FAIL"}
    assert decision == "CONDITIONAL"
```

F.6 Criteri di accettazione del package

Un package può essere dichiarato coerente con la presente monografia solo se:

- tutte le suite baseline passano sul namespace definitivo del package;
- ogni requisito congelato resta coperto almeno da uno scenario eseguibile;
- nessuna fixture critica dipende da servizi esterni non congelati;
- il replay della macchina a stati usato in validazione coincide con quello usato dal runtime;
- i test di bundle incompleto falliscono esplicitamente e non in modo silenzioso.

Contratti API, topologia del package e ordine dei calcoli

G.1 Package finale del motore

Il package unico del motore deve essere organizzato in namespace stabili e semanticamente puliti:

```
ga_zone_engine/
  quant/
    data.py
    volatility.py
    covariance.py
    targets.py
    quantiles.py
    survival.py
    emission.py
  runtime/
    guards.py
    state_machine.py
    dispatcher.py
    orders.py
  validation/
    pipeline.py
    diagnostics.py
    metrics.py
    gates.py
  reports/
    builders.py
    templates.py
  registry/
    manifest.py
    hashes.py
    persistence.py
  tests/
```

...

La topologia non è cosmetica: rende impossibile confondere layer quantitativo, lifecycle runtime, validazione OOS e reporting.

G.2 Oggetti dati principali

Il documento congela i seguenti oggetti logici come minimo comune denominatore del package.

Oggetto	Campi obbligatori minimi	Funzione
MarketEvent	event_time, event_type, bid, ask, last, volume, metadata	rappresenta l'evento osservato dal runtime
FeatureTensor	issued_at, instrument_id, shape, channels, values, hash	contiene X_t all'emissione
ConditionalRiskState	sigma_local, H_t, sigma_sys, psd_ok, fallback_used	riassume lo stato di rischio stimato
StructuralTargetCandidate	target_id, family, bounds, score_struct, score_liq	rappresenta un target del catalogo
PendingSignalDraft	instrument_id, side, entry_bounds, target_bounds, valid_from, valid_until, invalidation_price, risk_snapshot	bozza del contratto prima della persistenza
PendingSignal	tutti i campi core + state, touch_count, reason_code, core_hash	contratto runtime persistito
TrainingArtifact	modelli fitted, preprocessors, schema feature, soglie, metadati fold	artefatto di ricerca per un fold
InferenceBundle	manifest frozen completo, hash, versioni, policy fallback, runtime config	bundle unico usato per emissione e replay
ReplayLedgerRow	timestamp, stato precedente, stato successivo, evento, reason code, hash riferiti	audit trail evento per evento
FoldReport	metriche lifecycle, economiche, rischio, calibrazione, DSR, PBO, breakdown per regime	report di un fold outer

Oggetto	Campi obbligatori minimi	Funzione
GlobalValidationReport	aggregazioni multi-fold, decisione globale, warning, note di robustezza	esito finale della validazione

G.3 Contratti funzione del layer training

Il baseline del package congela le seguenti firme logiche:

```
def fit_training_bundle(config, train_dataset, valid_dataset, fold_spec) ->
    ↪ TrainingArtifact: ...

def fit_univariate_volatility(train_dataset, vol_family, exogenous=None): ...

def fit_cross_index_covariance(std_residuals, cov_family, fallback_policy): ...

def fit_quantile_layer(train_dataset, target_catalog, quantile_config): ...

def fit_pre_entry_survival(train_dataset, guard_config): ...

def fit_post_entry_survival(train_dataset, exit_config): ...
```

Ogni funzione deve essere pura rispetto ai propri input congelati: ogni dipendenza non dichiarata (feature implicite, configurazioni esterne, schemi colonne non persistiti) è vietata perché rompe la ricostruibilità del bundle.

G.4 Ordine dei calcoli nel training

L'ordine dei calcoli non è intercambiabile. Il baseline integrato impone la seguente sequenza:

1. partizionare temporalmente il fold con purge ed embargo già applicati;
2. fit del preprocessing solo sul train del fold;
3. fit dei modelli univariati di volatilità e generazione dei residui standardizzati;
4. fit del layer di covarianza cross-index sui residui standardizzati;
5. costruzione del tensore di feature coerente con lo schema persistito;
6. generazione del catalogo dei target strutturali candidati;
7. fit dei modelli di quantile per entry, target, durata e invalidazione;
8. fit dei modelli di survival pre-entry e post-entry;
9. replay su valid per valutare la fitness multi-obiettivo del cromosoma;
10. selezione evolutiva;
11. replay outer-test e costruzione del bundle frozen candidato.

Questa sequenza impedisce che modelli di quantile o survival siano fit prima di aver definito in modo coerente feature, rischio sistemico e catalogo target.

G.5 Contratti funzione del layer di emissione

```
def build_feature_tensor(latest_state, bundle) -> FeatureTensor: ...

def estimate_conditional_risk(feature_tensor, bundle) -> ConditionalRiskState: ...

def enumerate_structural_targets(target_catalog, feature_tensor, bundle): ...

def compute_zone_center(feature_tensor, target_candidate, bundle): ...

def compute_half_width(feature_tensor, risk_state, bundle): ...

def compute_valid_until(feature_tensor, target_candidate, bundle): ...

def compute_invalidation(center, half_width, risk_state, bundle): ...

def evaluate_publish_filters(feature_tensor, risk_state, target_candidate, bundle): ...

def persist_signal(signal_draft, bundle) -> PendingSignal: ...
```

Il layer di emissione deve restare privo di side effect non documentati: persiste solo il contratto finale scelto e il suo ledger di emissione, non candidati intermedi non pubblicati.

G.6 Contratti funzione del runtime

```
def on_market_event(signal, event, bundle): ...

def on_clock_event(signal, event, bundle): ...

def on_submit_order(signal, venue_response, bundle): ...

def on_order_event(signal, order_event, bundle): ...

def dispatch(signal, event, bundle): ...
```

La funzione `dispatch` è totale sul dominio degli eventi ammessi e deve applicare la precedenza congelata senza scorciatoie. Il runtime non può valutare probabilità nuove o rigenerare zone: consuma soltanto eventi e applica guardie.

G.7 Contratti funzione del layer validation

```
def replay_signal(signal, event_stream, bundle): ...

def replay_fold(test_dataset, bundle, fold_spec) -> FoldReport: ...

def compute_fold_metrics(replay_output, risk_metrics, calibration_metrics): ...
```

```
def compute_dsr(candidate_results, n_effective_trials): ...
```

```
def compute_pbo(candidate_matrix, scheme="CSCV"): ...
```

```
def classify_fold(fold_report, thresholds): ...
```

```
def classify_global(validation_reports, thresholds): ...
```

Il validatore deve usare la stessa `replay_signal` del runtime. Qualunque scorciatoia che sostituisca il replay con una stima approssimata rompe il principio di identità semantica fra ricerca e motore operativo.

G.8 Matrice delle dipendenze tra moduli

Modulo	Dipende da	Non può dipendere da
<code>quant.volatility</code>	dati train, preprocessors, config rischio	runtime, reportistica
<code>quant.covariance</code>	residui standardizzati, family config	dispatcher runtime
<code>quant.targets</code>	catalogo strutturale, feature tensor	stato ordine
<code>quant.quantiles</code>	feature tensor, target catalog	esiti runtime futuri
<code>quant.survival</code>	feature tensor, replay train/valid causalmente corretto	test dataset del fold
<code>runtime.guards</code>	bundle frozen, evento corrente, stato segnale	modelli fit live
<code>runtime.state_machine</code>	evento, guardie, precedenza	motore genetico
<code>validation.pipeline</code>	replay runtime, metriche, diagnostics	servizi di venue real-time
<code>reports.builders</code>	fold report, global report	modelli fit non persistiti
<code>registry.manifest</code>	bundle frozen, hash, versione	dati di mercato grezzi non riferiti

Schemi machine-readable, ledger e reporting unificato

H.1 Frozen manifest JSON

Ogni bundle frozen deve poter essere materializzato in un manifesto machine-readable di forma minima:

```
{
  "bundle_id": "uuid",
  "spec_version": "4.0",
  "model_long": {"path": "...", "sha256": "..."},
  "model_short": {"path": "...", "sha256": "..."},
  "preprocessor": {"path": "...", "sha256": "..."},
  "feature_schema": {"path": "...", "sha256": "..."},
  "risk_layer": {"vol_family": "EGARCH", "cov_family": "DCC"},
  "runtime_config": {"execution_window": "...", "guard_policy": "..."},
  "validation": {"dsr": 0.0, "pbo": 0.0},
  "environment": {"package_version": "...", "seed": 0}
}
```

Il manifesto non sostituisce gli artefatti; li collega in modo auditabile e rende verificabile l'assenza di componenti mancanti.

H.2 Schema record del segnale persistito

```
{
  "signal_id": "uuid",
  "instrument_id": "...",
  "side": "LONG",
  "issued_at": "...",
  "valid_from": "...",
  "valid_until": "...",
  "entry_zone": [0.0, 0.0],
  "target_zone": [0.0, 0.0],
}
```

```

    "invalidation_price": 0.0,
    "sigma_issued": 0.0,
    "bundle_id": "uuid",
    "feature_snapshot_hash": "...",
    "core_hash": "...",
    "state": "PENDING"
  }

```

H.3 Schema minimale del ledger runtime

Il ledger evento per evento deve essere esportabile in forma tabellare o CSV con le seguenti colonne minime:

Colonna	Significato
event_time	timestamp dell'evento consumato
event_type	tipo evento (MARKET_EVENT, CLOCK_EVENT, ORDER_EVENT, ...)
signal_id	riferimento al contratto
state_before	stato precedente
state_after	stato successivo
reason_code	motivazione della transizione
bundle_id	bundle frozen usato
feature_hash	snapshot delle feature riferite
core_hash	impronta del contratto
order_id	riferimento all'ordine, se presente

H.4 Schema del report di fold

Il report di fold deve esporre almeno quattro blocchi distinti:

1. **lifecycle metrics**: emission count, raw touch rate, executable rate, invalidation rate, missed target rate, executed-to-target hit rate;
2. **economic and risk metrics**: expected net return, CVaR, drawdown, ulcer index, MAE/MFE;
3. **calibration metrics**: Brier, reliability bins, calibration error;
4. **robustness metrics**: breakdown per regime, per bucket di correlazione, per liquidità, DSR, PBO, gate del fold.

H.5 Schema del report globale

Il report globale deve aggregare tutti i fold outer e includere:

- decisione finale (GO, CONDITIONAL, NO-GO);
- DSR globale e PBO globale;
- numero effettivo di prove usato per il DSR;

- lista warning sistematici per regime;
- confronto fra metriche aggregate e peggior fold;
- riferimento al bundle frozen scelto o al motivo di rifiuto.

H.6 Template minimo del report regime-by-regime

Per ogni regime il report deve fornire almeno:

Regime	Emissioni	Executable rate	Π^{E2E} empirica	Return netto medio	CVaR	Warning
Calmo						
Turbolento						
Contagio						
cross- index						
Illiquido						

Questo formato rende leggibile il vero comportamento del motore: non basta sapere che il totale OOS è buono; bisogna vedere se collassa esattamente nel regime che interessa evitare.

Casi di failure che la specifica esclude esplicitamente

La rifusione integrale non è completa se non dice anche cosa rifiuta. I failure mode che il documento esclude in modo esplicito sono i seguenti:

- **bundle non ricostruibile:** un motore privo di modello persistito, preprocessor fitted, schema feature e soglie congelate non è validabile in senso forte;
- **ricerca e runtime semanticamente divergenti:** se il validatore usa una semantica diversa dal dispatcher runtime, il backtest non ha valore causale;
- **touch inferiti da gap o da prezzi favorevoli ma non negoziabili:** i risultati così ottenuti sono metodologicamente nulli;
- **aggiornamento live dei campi core:** qualunque refresh di entry, target, invalidazione o finestra temporale dopo l'emissione è trattato come violazione strutturale;
- **stima del rischio non PSD o numericamente instabile senza fallback dichiarato:** il segnale non è deployable;
- **metriche aggregate senza breakdown di regime:** un risultato OOS non disaggregato per regime, liquidità e correlazione non è sufficiente a promuovere il motore.

Questa lista chiude il documento anche in senso negativo: non definisce solo ciò che il motore deve fare, ma anche ciò che non può più essere considerato ammissibile.

Manifest del frozen bundle, lessons learned e contratto di industrializzazione forte

J.1 Manifest obbligatorio del bundle frozen

Ogni bundle frozen del motore deve serializzare almeno i seguenti artefatti, ciascuno con hash forte, versione e timestamp di produzione:

Categoria	Artefatto obbligatorio	Scopo
Modelli	modello long, modello short o equivalente multi-side	ricostruire la logica di inferenza
Preprocessing	stato fitted del preprocessor	garantire trasformazioni identiche tra research e replay
Schema dati	ordine e schema delle feature persistite	impedire mismatch di colonne
Layer rischio	famiglia, parametri stimati e fallback policy di volatilità/covarianza	ricostruire σ_t e H_t
Layer quantile/survival	parametri e versioni dei modelli di quantile e hazard	ricostruire entry, durata, invalidazione e probabilità
Runtime	configurazione guardie, policy di conflitto, execution window, cancel policy	garantire identità tra replay e runtime
Validazione	metriche aggregate, report per regime, DSR, PBO, seed e configurazioni dei fold	audit OOS completo

Categoria	Artefatto obbligatorio	Scopo
Ambiente	versione package, dipendenze, hash del codice, seme random, data snapshot id	riproducibilità computazionale

J.2 Lezione industriale incorporata nella specifica

La specifica integra una lezione operativa che non viene più trattata come nota organizzativa ma come vincolo formale: **un modello non è OOS-validatable in senso forte se non esiste un frozen bundle completo**. In particolare, l'assenza di uno qualunque tra modello lato long/short, stato fitted del preprocessor, schema/ordine delle feature, soglie di inference e filtri/exit rules persistiti rende impossibile ricostruire in modo affidabile il frontier congelato.

Ne segue la regola forte:

Nessun candidato può essere dichiarato validato, né può essere promosso a run storico controllato, se il suo bundle frozen non consente replay deterministico entro tolleranza definita.

J.3 Hash chain e ledger

Il bundle frozen deve essere collegato a ogni segnale emesso tramite una hash chain minima:

$$H_{signal} = Hash(H_{bundle} \parallel H_{features} \parallel H_{core} \parallel issued_at).$$

Questa catena permette di risalire dall'evento runtime al bundle di ricerca che lo ha generato, impedendo sovrapposizioni ambigue tra versioni diverse del motore.

J.4 Contratto di replay deterministico

Per essere considerato industrialmente maturo, il sistema deve superare un replay deterministico con le seguenti proprietà:

- stessi dati di input implicano stessi segnali emessi;
- stessi segnali emessi implicano stesse transizioni della state machine;
- stessi eventi di ordine implicano stessi esiti del lifecycle segnale/ordine;
- differenze oltre tolleranza sono trattate come failure del bundle, non come “naturale variabilità”.

J.5 Boundary tra package scientifico e motore operativo

Il package reale può essere organizzato in namespace diversi (`quant`, `runtime`, `validation`, `reports`), ma il manifest frozen deve attraversarli tutti. In altre parole, la separazione modulare del codice non autorizza una separazione semantica del bundle. Il bundle frozen resta l'oggetto unico di ricostruibilità del motore.

J.6 Manifest documentale dei diagrammi e della build

Per rendere ricostruibile anche il layer documentale, la build hard-locked congela i seguenti artefatti grafici e di compilazione:

- sorgente canonica dei diagrammi in sintassi Mermaid (`.mmd`);
- asset statico integrato nel PDF (`.png`) corrispondente alla stessa logica del diagramma;
- versione target del renderer documentale: `mermaid@11.14.0`;
- theme target: `neutral`;
- pipeline di compilazione del documento: `pandoc + xelatex`;
- manifest JSON del bundle documentale contenente hash e parametri di build.

La versione distribuita del PDF integra direttamente gli asset statici congelati. Questo garantisce che il documento sia leggibile anche in assenza del renderer nativo, senza rinunciare alla ricostruibilità della logica dei diagrammi.

Riferimenti tecnici prioritari

La monografia recepisce la linea teorica classica e avanzata richiamata nelle fonti integrate. Per la versione theoretical hard-locked, i riferimenti principali vengono anche elencati in forma bibliografica sintetica così da rendere il baseline autosufficiente.

K.1 Bibliografia puntuale delle estensioni hard-locked

1. Engle, R. F. (1982). *Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation*. *Econometrica*.
2. Bollerslev, T. (1986). *Generalized Autoregressive Conditional Heteroskedasticity*. *Journal of Econometrics*.
3. Nelson, D. B. (1991). *Conditional Heteroskedasticity in Asset Returns: A New Approach*. *Econometrica*.
4. Glosten, L. R., Jagannathan, R., Runkle, D. E. (1993). *On the Relation Between the Expected Value and the Volatility of the Nominal Excess Return on Stocks*. *Journal of Finance*.
5. Hansen, P. R., Huang, Z., Shek, H. H. (2012). *Realized GARCH: A Joint Model for Returns and Realized Measures of Volatility*. *Journal of Applied Econometrics*.
6. Bollerslev, T. (1990). *Modelling the Coherence in Short-Run Nominal Exchange Rates: A Multivariate Generalized ARCH Model*. *Review of Economics and Statistics*.
7. Engle, R. F. (2002). *Dynamic Conditional Correlation: A Simple Class of Multivariate GARCH Models*. *Journal of Business & Economic Statistics*.
8. Engle, R. F., Kroner, K. F. (1995). *Multivariate Simultaneous Generalized ARCH*. *Econometric Theory*.
9. Cappiello, L., Engle, R. F., Sheppard, K. (2006). *Asymmetric Dynamics in the Correlations of Global Equity and Bond Returns*. *Journal of Financial Econometrics*.
10. Aielli, G. P. (2013). *Dynamic Conditional Correlation: On Properties and Estimation*. *Journal of Business & Economic Statistics*.
11. Morana, C. (2015). *Semiparametric Estimation of Multivariate GARCH Models*. *Journal of Econometrics / working-paper lineage cited in Deep Research*.
12. Morana, C. (2019). *Regularized Semiparametric Estimation of High Dimensional Dynamic*

- Conditional Covariance Matrices*. Econometric Reviews / working-paper lineage cited in Deep Research.
13. Koenker, R., Bassett, G. (1978). *Regression Quantiles*. Econometrica.
 14. Cox, D. R. (1972). *Regression Models and Life-Tables*. Journal of the Royal Statistical Society, Series B.
 15. Fine, J. P., Gray, R. J. (1999). *A Proportional Hazards Model for the Subdistribution of a Competing Risk*. Journal of the American Statistical Association.
 16. Rockafellar, R. T., Uryasev, S. (2000). *Optimization of Conditional Value-at-Risk*. Journal of Risk.
 17. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T. (2002). *A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II*. IEEE Transactions on Evolutionary Computation.
 18. Bailey, D. H., López de Prado, M. (2014). *The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality*.
 19. Bailey, D. H., Borwein, J., López de Prado, M., Zhu, Q. J. (2017). *The Probability of Backtest Overfitting*. Journal of Computational Finance.
 20. White, H. (2000). *A Reality Check for Data Snooping*. Econometrica.
 21. Hansen, P. R. (2005). *A Test for Superior Predictive Ability*. Journal of Business & Economic Statistics.
 22. Newey, W. K., West, K. D. (1987). *A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix*. Econometrica.
 23. Politis, D. N., Romano, J. P. (1994). *The Stationary Bootstrap*. Journal of the American Statistical Association.

K.2 Famiglie metodologiche coperte dal documento

- ARCH, GARCH, EGARCH, GJR-GARCH e Realized GARCH per la varianza condizionale;
- CCC, DCC, ADCC, cDCC e BEKK per la dipendenza condizionale multi-indice;
- benchmark semiparametrici SP-DCC/RSP-DCC per shrinkage avanzato in large-N;
- quantile regression per geometria di entry, target e durata;
- Cox, discrete-time hazard e competing risks per la decomposizione del lifecycle;
- CVaR per il controllo del tail risk;
- NSGA-II con dominance vincolata per la ricerca evolutiva multi-obiettivo;
- DSR, PBO, White Reality Check e Hansen SPA per la governance anti-overfitting.

Nota d'uso del documento

Il documento qui generato è stato pensato per essere usato come nuova base di riferimento. Non modifica i file sorgente e non dipende dalla forma conversazionale dei PDF originali. Se deve sostituire l'attuale baseline integrata breve o la successiva rifusione unica, la sostituzione corretta è utilizzare questo nuovo PDF hard-locked come base ufficiale.